PEOPLE'S DEMOCRATIC REPUBLIC OF ALGERIA

الجمهورية الجزائرية الديمقراطية الشعبية

MINISTRY OF HIGHER EDUCATION AND SCIENTIFIC RESEARCH

وزارة التعليم العالي والبحث العلمي

École supérieure en Sciences et Technologies de l'Informatique et du Numérique

المدرسة العليا في علوم وتكنولوجيات الإعلام الآلي بجاية



Dissertation Submitted to the Department of 2nd Cycle Training in Partial Fulfillment of the Requirements for Engineer's Degree in Computer Science

Specialty: Cyber Security

Submitted by:

ABDELLI Amira Farah

ProvSAGE: A Lightweight Spatio-Temporal GNN for Provenance-Based Host Intrusion Detection

Supervised by:

Dr. AIT TALEB Samiha – ESTIN

Mr. ABDELLI Nasreddine – ESTIN

Members of jury:

• **Dr.** BAZIZI Lydia	President	ESTIN
• **Dr.** DJEDOUI Layachi	Examiner	ESTIN
• **Ms.** ADRAR Amal	Examiner	ESTIN

Academic year: 2025/2026

Dedication

To my family, for their unwavering love, encouragement, and endless support.

To my parents, I cannot express how grateful I am for your constant presence, guidance, and sacrifices throughout this five-year journey, filled with both challenges and achievements. This accomplishment would not have been possible without you.

To my grandmother, thank you for always believing in me, for reminding me of my potential, and for encouraging me to believe in myself, even during the moments when I struggled to do so.

To my siblings, Yacine, Amina, Kamel, Yasmine, and Khadidja, thank you for always standing by my side, for your support during difficult times, and for being a constant source of inspiration.

To Narimane and Koussaila, thank you for your presence, your kindness, and for bringing so much light and warmth to our family.

To my nephews, Mayas, Danyl, and Zayn, thank you for being the rays of sunshine that brighten my days and fill my life with joy.

To Katia and Rayan, thank you for your friendship since our first years of school. Your constant support, encouragement, and loyalty have always reminded me that I have people I can count on.

To Nesrine, Meriem, Lamia, Lilia and all my cousins, aunts, and uncles, thank you for your love, encouragement, and support throughout the years.

To all my friends, and to everyone who has been there for me, thank you for your presence, your kindness, and for contributing, in one way or another, to the person I have become today.

Finally, thank you to every person who has been part of my journey. I am deeply grateful for the love, support, and memories you have given me, and for helping make life brighter along the way.

Acknowledgements

I would like to express my sincere gratitude to my supervisors, Dr. Samiha Ait Taleb and M. Nasreddine Abdelli, for their guidance, patience, and invaluable advice throughout this project. Their expertise, encouragement, and continuous support were essential to the successful completion of this work.

I would also like to extend my appreciation to the members of the jury for accepting to evaluate this thesis and for the time and effort they dedicate to reviewing this work.

My heartfelt thanks go to the faculty members and staff of the department for providing a stimulating learning environment and for sharing their knowledge and experience throughout my academic journey.

I am deeply grateful to my family for their unconditional love, unwavering support, and constant encouragement throughout my years of study. Their belief in me has been a source of strength and motivation during both challenging and rewarding moments.

I would also like to thank my friends and colleagues who accompanied me throughout this journey, offering support, advice, and encouragement whenever needed.

Finally, I would like to express my gratitude to everyone who contributed, directly or indirectly, to the completion of this thesis. Your support and kindness have been greatly appreciated and will always be remembered.

Thank you all.

Abstract

Advanced Persistent Threats (APTs) on enterprise endpoints unfold as long causal chains of system operations that elude signature-based and rule-based detectors. Reasoning over system provenance graphs, which capture the causal history of all system interactions, has emerged as the principled response to this limitation. STGAN is a recent learning-based detector built on this paradigm that fuses structural, semantic, and temporal features to reach state-of-the-art performance on the DARPA TRACE benchmark. Its ablation study isolates the contribution of each module and tests two architectural alternatives, but does not extend the comparison to semantic embeddings beyond Word2Vec, to neighborhood-aggregation backbones such as GraphSAGE, or to non-attention fusion modules.

This engineering project introduces ProvSAGE, a lightweight variant of STGAN derived from three empirical contributions on DARPA TRACE. The first contribution is a comparative study of semantic embedding alternatives to Word2Vec. The second contribution is a comparative study of architectural alternatives within the graph neural network backbone and the fusion module. The third contribution is a post-detection investigation module that compresses STGAN's flag set into a compact attack-neighborhood subgraph by combining entity-type filtering, multi-window temporal scoping, and topological-compactness ranking, without consulting ground-truth labels. The recommended ProvSAGE configuration improves detection accuracy over STGAN while substantially reducing both the parameter count and the training time. All experiments use deterministic training and paper-faithful neighbour sampling, ensuring reproducibility.

Keywords: provenance graphs, intrusion detection, Advanced Persistent Threats, graph neural networks, GraphSAGE, ProvSAGE.

Résumé

Les menaces persistantes avancées (APT) sur les postes de travail se déploient en longues chaînes causales d'opérations système qui échappent aux détecteurs par signature et par règles. L'analyse des graphes de provenance système, qui capturent l'historique causal de toutes les interactions, s'est imposée comme la réponse de principe à cette limite. STGAN est un détecteur récent fondé sur l'apprentissage qui, dans ce paradigme, atteint des performances de pointe sur le jeu de données DARPA TRACE en fusionnant des caractéristiques structurelles, sémantiques et temporelles. Son étude d'ablation isole la contribution de chaque module et teste deux alternatives architecturales, mais n'étend pas la comparaison aux embeddings sémantiques au-delà de Word2Vec, aux backbones à agrégation de voisinage tels que GraphSAGE, ni aux modules de fusion sans attention.

Ce projet de fin d'études d'ingénieur introduit ProvSAGE, une variante allégée de STGAN issue de trois contributions empiriques sur DARPA TRACE. La première contribution est une étude comparative des alternatives d'embeddings sémantiques au Word2Vec. La deuxième contribution est une étude comparative des alternatives architecturales au sein du réseau de neurones sur graphe et du module de fusion. La troisième contribution est un module d'investigation post-détection qui compresse l'ensemble des nœuds flaggués par STGAN en un sous-graphe compact représentant le voisinage d'attaque candidat, en combinant un filtrage par type d'entité, une recherche multi-fenêtres de densité temporelle et un classement par compacité topologique, sans recourir aux étiquettes de vérité terrain. La configuration ProvSAGE recommandée améliore la précision de détection par rapport à STGAN tout en réduisant substantiellement à la fois le nombre de paramètres et le temps d'entraînement. Toutes les expériences utilisent un entraînement déterministe et l'échantillonnage de voisinage du papier d'origine, ce qui garantit la reproductibilité.

Mots-clés : graphes de provenance, détection d'intrusions, menaces persistantes avancées, réseaux de neurones sur graphes, GraphSAGE, ProvSAGE.

الملخص

تشكل التهديدات المستمرة المتقدمة (APT) على نقاط النهاية المؤسسية سلاسل سببية طويلة من عمليات النظام تفلت من أنظمة الكشف القائمة على التواقيع أو القواعد. ظهر تحليل رسومات المصدر، التي تلتقط التاريخ السببي لجميع تفاعلات النظام، استجابة مبدئية لهذه المحدودية. يحقق STGAN، وهو كاشف حديث قائم على التعلم ضمن هذا النموذج، أداء يضاهي أحدث ما توصل إليه المجال على مجموعة بيانات TRACE DARPA من خلال دمج خصائص بنيوية ودلالية وزمنية. تعزل دراسة الاجتثاث الخاصة به مساهمة كل وحدة وتختبر بديلين معماريين، لكنها لا توسع المقارنة لتشمل التمثيلات الدلالية بخلاف Word2Vec، أو شبكات تجميع الجوار مثل GraphSAGE، أو وحدات الدمج غير المعتمدة على الانتباه.

يقدم هذا المشروع ProvSAGE، وهو نسخة مخففة من STGAN مستخلصة من ثلاث مساهمات تجريبية على TRACE. DARPA المساهمة الأولى دراسة مقارنة لبدائل التمثيلات الدلالية بدلا من Word2Vec. المساهمة الثانية دراسة مقارنة للبدائل المعمارية ضمن الشبكة العصبية على الرسم ووحدة الدمج.المساهمة الثالثة وحدة للتحقيق ما بعد الكشف تضغط مجموعة العقد التي رصدها STGAN إلى رسم فرعي مدمج يمثل جوار الهجوم المرشح، عبر تصفية حسب نوع الكيان، وبحث متعدد النوافذ عن كثافة زمنية، وترتيب حسب التراص الطوبولوجي، دون الاستعانة بتسميات الحقيقة المرجعية. تحسن التهيئة الموصى بها لـ ProvSAGE دقة الكشف مقارنة بـ STGAN مع تقليل كبير في كل من عدد المعاملات وزمن التدريب. تستخدم جميع التجارب تدريبا حتميا وأخذ عينات الجوار وفق الورقة الأصلية، مما يضمن إمكانية إعادة الإنتاج.

الكلمات المفتاحية: رسومات المصدر، كشف الاختراقات، التهديدات المستمرة المتقدمة، الشبكات العصبية على الرسم، ProvSAGE GraphSAGE،

Contents

Abstract iii

Résumé iv

الملخص v

List of Abbreviations x

List of Figures xii

List of Tables xiii

General Introduction 1

1 Fundamentals of Cybersecurity and Provenance-Based Detection 4
- 1.1 Introduction . . . 4
- 1.2 Endpoint Security and APTs . . . 4
  - 1.2.1 Endpoint security landscape . . . 4
  - 1.2.2 Advanced Persistent Threats: lifecycle and characteristics . . . 5
  - 1.2.3 Why traditional defenses fall short . . . 6
- 1.3 System Provenance Graphs . . . 7
  - 1.3.1 Definition and formal model . . . 7
  - 1.3.2 Construction from operating-system audit logs . . . 7
  - 1.3.3 Granularity, scale, and reduction techniques . . . 8
- 1.4 Graph Learning for Security . . . 9
  - 1.4.1 Message passing on heterogeneous graphs . . . 9
  - 1.4.2 Embedding-based representations of system entities . . . 9
- 1.5 Conclusion . . . 10

2 Context and Problem Statement 11
- 2.1 Introduction . . . 11
- 2.2 The STGAN Model: Promise and Open Questions . . . 11
  - 2.2.1 Why STGAN was selected as the baseline . . . 11
  - 2.2.2 The role of the semantic stream . . . 12

2.2.3 The role of the structural and fusion components . . . 12
2.3 Problem Statement . . . 13
2.3.1 The impact of the embedding choice . . . 13
2.3.2 The contribution of attention-based components . . . 13
2.3.3 The structure of STGAN's anomaly set for investigation . . . 14
2.4 Project Objectives . . . 15
2.4.1 Reproduction of the baseline . . . 15
2.4.2 Comparative study of semantic embeddings . . . 15
2.4.3 Architectural ablation . . . 15
2.4.4 Investigation study . . . 16
2.4.5 Reproducibility and practical demonstration . . . 16
2.5 Conclusion . . . 17

3 State of the Art: Learning on Provenance Graphs 18
3.1 Introduction . . . 18
3.2 Detection Systems Based on Provenance Graphs . . . 18
3.2.1 Static and snapshot-based approaches . . . 18
3.2.2 Temporal and dynamic approaches . . . 19
3.2.3 Position of STGAN in this landscape . . . 19
3.3 Semantic Representations of System Entities . . . 20
3.3.1 From bag-of-words to learned embeddings . . . 20
3.3.2 Static word embeddings . . . 20
3.3.3 Contextual transformer embeddings . . . 21
3.4 Lightweight Graph Neural Networks . . . 21
3.4.1 Trade-offs between attention and aggregation . . . 21
3.4.2 Position of GraphSAGE and GCN . . . 21
3.5 Remaining Gaps Addressed in This Work . . . 22
3.6 Conclusion . . . 22

4 Technical Foundations: STGAN, Dataset and Embedding Models 23
4.1 Introduction . . . 23
4.2 The STGAN Architecture . . . 23
4.2.1 Overview: three streams and a fusion . . . 23
4.2.2 Structural stream: Graph attention network . . . 24
4.2.3 Semantic stream: Word2Vec and positional encoder . . . 24
4.2.4 Temporal stream: Temporal graph network . . . 24
4.2.5 Fusion and classification: multi-head self-attention . . . 25
4.2.6 Training procedure and self-distillation mask . . . 25
4.3 The DARPA TRACE Dataset . . . 25
4.3.1 Origin and motivation . . . 25

4.3.2 The CDM18 schema and the raw JSON format . . . 25
4.3.3 Train/test split, ground truth, and statistics . . . 26
4.4 Semantic Embedding Families . . . 26
4.4.1 Word2Vec: static word-level embedding . . . 26
4.4.2 FastText: static sub-word embedding . . . 26
4.4.3 BERT: contextual transformer embedding . . . 27
4.4.4 EmbeddingGemma: task-aware contextual embedding . . . 27
4.5 Lightweight Graph Neural Networks Considered . . . 27
4.5.1 GraphSAGE: aggregation without attention . . . 27
4.5.2 GCN: graph convolution . . . 27
4.6 Conclusion . . . 28

5 ProvSAGE: Design and Methodology 29
5.1 Introduction . . . 29
5.2 Reproduction of the STGAN Baseline . . . 29
5.2.1 Hardware and software environment . . . 30
5.2.2 Data preparation pipeline . . . 30
5.2.3 Memory constraints and neighbour-sampling . . . 30
5.2.4 Reproduction result and gap with the published figure . . . 31
5.3 Comparative Embedding Study . . . 31
5.3.1 Motivation and hypothesis . . . 31
5.3.2 Integration of the four families into STGAN . . . 32
5.3.3 Linear projection for contextual embeddings . . . 32
5.3.4 Caching of frozen contextual embeddings . . . 33
5.3.5 Configurations evaluated . . . 33
5.3.6 Experimental protocol . . . 33
5.4 Architectural Ablation . . . 34
5.4.1 Motivation and hypothesis . . . 34
5.4.2 Alternatives to the graph attention network . . . 34
5.4.3 Alternatives to the multi-head fusion . . . 35
5.4.4 Configurations evaluated . . . 35
5.4.5 Parameter-count and timing instrumentation . . . 35
5.4.6 Experimental protocol . . . 36
5.5 Attack Neighborhood Investigation . . . 36
5.5.1 Motivation and forensics framing . . . 36
5.5.2 ProvSAGE-C3 pipeline . . . 37
5.5.3 Validation methodology . . . 41
5.6 Implementation Notes . . . 42
5.6.1 Code organisation and reproducibility . . . 42

5.6.2 Optimisation of the evaluation pipeline . . . 42
5.6.3 Determinism and reproducibility safeguards . . . 43
5.7 Conclusion . . . 44

6 Experimental Evaluation and Results **45**
6.1 Introduction . . . 45
6.2 Reproduction Results . . . 45
6.2.1 Detection metrics on DARPA TRACE . . . 46
6.2.2 Comparison with the published figure . . . 46
6.3 Comparative Embedding Study Results . . . 47
6.3.1 Detection metrics across the five configurations . . . 47
6.3.2 Pair-wise comparison of configurations . . . 48
6.3.3 Shared failure modes and ground-truth overlap . . . 49
6.3.4 Interpretation . . . 49
6.4 Architectural Ablation Results . . . 50
6.4.1 Detection metrics across the five configurations . . . 50
6.4.2 Parameter-count and complexity analysis . . . 51
6.4.3 Best configuration: GraphSAGE with multi-head fusion . . . 51
6.4.4 Interpretation . . . 52
6.5 Investigation Study Results . . . 52
6.5.1 Pipeline output and quantitative metrics . . . 53
6.5.2 Visualization of the candidate attack neighborhood . . . 55
6.5.3 Interpretation . . . 56
6.6 Synthesis . . . 57
6.7 Conclusion . . . 59

General Conclusion & Perspectives **60**

References **62**

List of Abbreviations

- **ACM** – Association for Computing Machinery
- **APT** – Advanced Persistent Threat
- **BERT** – Bidirectional Encoder Representations from Transformers
- **CDM** – Common Data Model (CDM18 schema, DARPA Transparent Computing)
- **CPU** – Central Processing Unit
- **CUDA** – Compute Unified Device Architecture (NVIDIA)
- **DARPA** – Defense Advanced Research Projects Agency
- **eBPF** – extended Berkeley Packet Filter
- **EDR** – Endpoint Detection and Response
- **ETW** – Event Tracing for Windows
- **F1** – F1 score (harmonic mean of precision and recall)
- **FN** – False Negative
- **FP** – False Positive
- **GAT** – Graph Attention Network
- **GCN** – Graph Convolutional Network
- **GNN** – Graph Neural Network
- **GPU** – Graphics Processing Unit
- **GRU** – Gated Recurrent Unit
- **JSON** – JavaScript Object Notation
- **MHA** – Multi-Head Attention
- **MIG** – Multi-Instance GPU
- **NVML** – NVIDIA Management Library
- **STGAN** – Spatio-Temporal Graph Attention Network

- **TextRCNN** – Text Recurrent Convolutional Neural Network
- **TGN** – Temporal Graph Network
- **TN** – True Negative
- **TP** – True Positive
- **TRACE** – DARPA Transparent Computing TRACE engagement dataset
- **UUID** – Universally Unique Identifier
- **WWW** – World Wide Web (ACM Web Conference)

List of Figures

4.1 Architecture of STGAN 24

5.1 CDM18 entity type mapping 30
5.2 BERT linear projection 33
5.3 ProvSAGE-C3 pipeline overview 37
5.4 ProvSAGE-C3 pipeline: detection-side processing (Steps 0 through 5). 40
5.5 ProvSAGE-C3 pipeline: baseline comparison and persistence (continuation). . 41
5.6 Deterministic setup routine 43

6.1 `helper()` function of the original STGAN code. 54
6.2 Console output of ProvSAGE-C3 on DARPA TRACE 55
6.3 The candidate attack neighborhood returned by ProvSAGE-C3 on DARPA TRACE. 56

List of Tables

3.1 Position of representative provenance-based detection systems along the three axes used to organize the chapter. 20

6.1 Detection metrics of the STGAN baseline (Word2Vec + GAT + Multi-Head Attention) reproduced under deterministic conditions on DARPA TRACE. 46
6.2 Evaluation of the publicly released STGAN weights on the DARPA TRACE test set with an independently trained Word2Vec. 47
6.3 Detection metrics of the five semantic embedding configurations on DARPA TRACE. All configurations use the GAT + Multi-Head Attention architecture inherited from the published STGAN model. 48
6.4 Detection metrics of the five architectural configurations on DARPA TRACE. The semantic embedding is FastText in every row. 50
6.5 Output of the multi-window temporal scoping stage of ProvSAGE-C3 on the DARPA TRACE flag set. The compactness criterion ranks candidates by ascending expansion size; the top-ranked candidate is the predicted attack neighborhood. 53
6.6 Interaction matrix between the embedding choice and the architectural choice on DARPA TRACE, restricted to the two embeddings and the two architectures relevant to ProvSAGE. 58

General Introduction

Over the past two decades, the digital activity of enterprise systems has grown into a continuous stream of low-level events. These events include process creations, file accesses, network connections, and system calls, all recorded as audit logs. Within this stream, sophisticated adversaries known as Advanced Persistent Threats (APTs) conduct multi-stage intrusion campaigns that may unfold over weeks or months. Each individual operation in such a campaign appears, in isolation, indistinguishable from legitimate system activity; only the chain of causally related events reveals the attack. Traditional defenses such as signature-based antivirus, rule-driven host intrusion detection systems, and log-correlation security platforms operate at the level of single events or short windows. They are therefore systematically blind to the long-range causal patterns that characterize APT campaigns.

The provenance-graph paradigm has emerged as a response to this limitation. A provenance graph is a directed, heterogeneous graph constructed from audit logs. Its nodes represent system entities such as processes, files, network sockets, and memory regions. Its edges represent the timestamped actions linking them. In this representation, an attack manifests as a connected subgraph whose nodes and edges encode the causal trace of the intrusion. The detection problem reduces to a node classification task: for each entity, the system decides whether it participates in malicious activity on the basis of its structural position in the graph, its semantic identity, and the temporal evolution of its interactions.

Among the recent learning-based systems built on this paradigm, STGAN (Sang et al., 2025) combines three streams of information through a multi-head self-attention fusion module. The structural stream applies a graph attention network over the provenance graph topology. The semantic stream applies Word2Vec to entity-level token sequences. The temporal stream applies a temporal graph network to capture the dynamics of node interactions. The fused representation is then passed to an XGBoost classifier that predicts each node's type, and entities whose prediction deviates from their declared type are flagged as anomalous. The published evaluation reports an F1 score of 0.9916 on the DARPA TRACE dataset and includes an ablation study that isolates the contribution of each module and tests two architectural alternatives: a graph convolutional network in place of the graph attention network, and a single-head self-attention layer in place of the multi-head attention. The comparison space is not extended, however, to semantic

embeddings beyond Word2Vec, to neighborhood-aggregation backbones such as GraphSAGE, or to non-attention fusion modules such as pooling and averaging.

This engineering project extends STGAN's comparison space and identifies a configuration that is simultaneously more accurate and more efficient than the published baseline. The resulting variant is named ProvSAGE, and three empirical contributions support its design. The first contribution is a comparative study of semantic embedding alternatives. The Word2Vec module is replaced by FastText, BERT, and EmbeddingGemma in two configurations, and the resulting detection performance is compared under a single controlled experimental protocol. The second contribution is a comparative study of architectural alternatives. The graph attention network is replaced by GraphSAGE or by a graph convolutional network, and the multi-head attention fusion is replaced by max-pooling or by averaging.The third contribution is a post-detection investigation module. Starting from the anchor set returned by STGAN, the module compresses the anchors into a compact subgraph representing the candidate attack neighborhood. The compression combines three signals: the entity type of each anchor, the temporal density of the anchor's edges, and the compactness of the resulting topological expansion. The module operates entirely on signals available at inference time and does not consult ground-truth labels in its decisions.

Overview of Main Results. Three salient findings emerge from the experimental evaluation. First, no alternative embedding outperforms Word2Vec under paper-faithful neighbour sampling, which suggests that the structural richness of the provenance graph already supplies the discriminative signal that richer embeddings would otherwise add. Second, replacing the graph attention network with GraphSAGE while preserving the multi-head attention fusion yields the recommended ProvSAGE configuration. This configuration reaches an F1 score of 0.9932 with a 14 % reduction in parameter count and a fivefold reduction in training time relative to STGAN, and the result is robust to the choice of semantic embedding. Third, the post-detection investigation module reduces STGAN's flag set on the DARPA TRACE dataset to a candidate attack-neighborhood subgraph of approximately 343 nodes, corresponding to a compression of three orders of magnitude relative to the full provenance graph. The candidate is validated against the ground truth and contains a majority of STGAN's correctly flagged anchors along with additional confirmed malicious entities recovered by 1-hop topological expansion.

Thesis Structure

The remainder of this document is organized as follows.

- **Chapter 1** establishes the fundamentals of cybersecurity needed to follow the work: endpoint security, the lifecycle of APT campaigns, the limits of traditional defenses, the principles of system provenance graphs, and an overview of graph neural networks.

- **Chapter 2** defines the practical context and the problem statement of the project: the STGAN system, the open questions it raises in light of its published ablation, and the concrete objectives pursued in this work.
- **Chapter 3** presents a focused state of the art on learning-based detection on provenance graphs and on the families of semantic embeddings considered in the rest of the document.
- **Chapter 4** details the technical foundations on which the project builds: the STGAN architecture, the DARPA TRACE dataset and its preparation pipeline, the four embedding models studied, and the alternative graph neural networks used in the architectural comparison.
- **Chapter 5** describes ProvSAGE: the reproduction of the baseline, the comparative embedding study, the comparative architectural study, the exploratory investigation study, and their implementation choices.
- **Chapter 6** reports the experimental evaluation and the resulting analysis: reproduction results, embedding study, architectural study, investigation study, and a final synthesis.
- The **general conclusion** synthesizes the contributions, discusses the limits encountered, and outlines avenues for future work.

Chapter 1

Fundamentals of Cybersecurity and Provenance-Based Detection

1.1 Introduction

This chapter establishes the foundations on which the rest of the thesis is built. It does not present new contributions. Instead, it provides the conceptual vocabulary and the technical background that a reader needs in order to understand the problem we address, the data we work with, and the techniques we ultimately compare.

Three notions are introduced in succession. The first is the modern endpoint-security setting and the class of threats that motivate the entire research effort: Advanced Persistent Threats. The second is the family of system provenance graphs, which has emerged as the principled data structure for analysing system-level activity at scale (Han et al., 2018). The third is the family of graph-learning techniques that operate on such graphs and that form the algorithmic backbone of the model studied in this work (Bilot et al., 2023). By the end of the chapter, the reader has the elements necessary to follow the more specific technical and methodological discussions of the subsequent chapters.

1.2 Endpoint Security and APTs

1.2.1 Endpoint security landscape

An *endpoint* is any device that connects to an organisation's information system and on which user or application code executes. Endpoints include workstations, laptops, servers, mobile devices, and, increasingly, virtualization compute units in cloud infrastructures. From a defensive perspective, endpoints occupy a privileged position. They are the location where most user-driven actions originate. They are also the location where confidential data is ultimately

read or written. They are, finally, the location where the majority of intrusions materialize into observable behavior.

Endpoint defense has progressed through several generations of tools. Signature-based antivirus engines, dominant until the early 2010s, match files against known malicious patterns. Host-based intrusion detection systems extended this paradigm by monitoring suspicious behavioral rules. The current generation, marketed as Endpoint Detection and Response (EDR), shifts the emphasis from prevention to continuous observation. EDR platforms collect fine-grained telemetry about process executions, file accesses, network connections, and inter-process communications. The premise of EDR is that even when an intrusion bypasses preventive controls, the actions of the attacker leave traces in the telemetry stream (Han et al., 2020). The challenge is then to recover those traces from a torrent of legitimate activity.

This shift in posture, from blocking known threats at the perimeter to detecting unknown ones inside the system, is the broader context for the line of research pursued in this thesis. The detection task addressed in the following chapters operates precisely on the kind of fine-grained, system-level telemetry that modern EDR platforms collect.

1.2.2 Advanced Persistent Threats: lifecycle and characteristics

Advanced Persistent Threats (APTs) designate long-running, goal-driven intrusion campaigns conducted by well-resourced adversaries against specific strategic targets (Milajerdi et al., 2019). The term captures three defining attributes. The campaigns are *advanced* in that they routinely combine custom tooling, zero-day exploits, and specialized tradecraft. They are *persistent*, often maintaining a foothold within the target environment for weeks, months, or even years. They are finally driven by an identifiable *threat* actor, such as a nation-state agency, a sophisticated criminal group, or a private operator pursuing economic or political objectives.

The behavior of an APT campaign is conventionally described as a sequence of stages. Frameworks such as the Cyber Kill Chain and MITRE ATT&CK differ in their taxonomy but agree on the overall shape. The attacker conducts reconnaissance, gains initial access, and establishes a command-and-control channel. The attacker then escalates privileges, harvests credentials, and moves laterally across the internal network. The attacker finally locates data of interest and exfiltrates or sabotages the target (Han et al., 2020). Stages may span weeks and may be interleaved with deliberate periods of inactivity intended to evade timing-based heuristics.

A critical observation for detection research is that, when examined in isolation, the individual operations executed within any single stage are often indistinguishable from legitimate administrative activity. A PowerShell invocation, a remote file copy, or a credential read each have benign counterparts. The malicious nature of an APT therefore emerges only when these operations are analyzed as a coherent sequence with causal and temporal relationships between them (Milajerdi et al., 2019). This property has a direct methodological consequence, explored

in the next subsection. It sets a lower bound on the kind of representation that a detection system must manipulate.

1.2.3 Why traditional defenses fall short

The defensive techniques inherited from earlier generations of endpoint security struggle against the APT model for three connected reasons.

Signature-based detection relies on matching observed artefacts against a database of known indicators of compromise. Such artefacts include file hashes, domain names, registry keys, and command-line patterns. The approach is effective against commodity malware whose components are reused at scale. APT actors, however, routinely employ custom binaries, ephemeral infrastructure, and living-off-the-land techniques in which only standard system tools are invoked. Each campaign produces a fresh set of artefacts, and signatures can only be written *after* a campaign has been identified and analyzed. Detection by signature is therefore structurally reactive.

Per-event anomaly detection attempts to flag deviations from a learned baseline of system activity. The approach can, in principle, catch novel behaviour. It suffers, however, from a poor signal-to-noise ratio on the kind of telemetry that modern endpoints generate. Legitimate administrative work routinely produces statistical outliers, and APT operations are deliberately crafted to remain within the envelope of normal activity. Alert fatigue rapidly degrades the practical utility of such systems (Han et al., 2018).

Rule-based behavioral detection encodes expert knowledge of attack patterns into explicit rules. It improves on signatures by expressing relationships between events rather than literal artefacts. Each rule, however, still describes a single, locally observable pattern. The defining feature of APTs, that maliciousness is a property of the causal sequence of operations spread across processes, files, and network endpoints, exceeds the expressive power of per-event or short-window rules (Milajerdi et al., 2019).

These limitations point to a structural requirement. A detection system capable of addressing APTs must reason over the causal and temporal relationships between events. It must do so at a granularity that preserves the individual entities involved, and over a temporal horizon long enough to capture the campaign as a whole. This requirement is precisely what system provenance graphs are designed to satisfy.

1.3 System Provenance Graphs

1.3.1 Definition and formal model

A system provenance graph is a structured representation of the causal relationships between events that occur on a host. Each node of the graph corresponds to a system-level entity. Each edge corresponds to a directly observed interaction between two entities. The entities typically include processes, files, network endpoints, memory regions, and kernel objects. The interactions include actions such as a process reading a file, sending data over a socket, forking a child process, or loading a shared library (Han et al., 2018).

Formally, a provenance graph can be modeled as a directed, attributed, heterogeneous, and temporally annotated graph $G = (V, E, \tau, \phi, \psi)$. The set V contains the entities observed during the monitoring window. The set $E \subseteq V \times V$ contains the causal events. The function $\tau : E \to R$ labels each edge with a timestamp. This timestamp encodes the order in which interactions occurred. The functions $\phi : V \to \mathcal{T}_V$ and $\psi : E \to \mathcal{T}_E$ assign each node and each edge a type drawn from finite vocabularies. Typical vocabularies are $\mathcal{T}_V = \{\text{PROCESS}, \text{FILE}, \text{NETWORK}, \ldots\}$ and $\mathcal{T}_E = \{\text{READ}, \text{WRITE}, \text{EXEC}, \ldots\}$.

The graph structure captures information that is intrinsically lost when events are examined in isolation. A single WRITE edge, taken alone, conveys little semantic content. The same operation occurs millions of times per hour during normal use. The same edge embedded in a longer causal chain, a freshly spawned process inheriting from a remote shell, which read a sensitive file, and which then wrote to a hidden directory, acquires a very different meaning. By preserving these long-range causal dependencies, provenance graphs provide the substrate over which sophisticated, multi-stage threats can be reasoned about.

1.3.2 Construction from operating-system audit logs

Building a provenance graph requires a stream of system-level events. This stream must be both *complete*, in the sense of capturing the operations relevant to detection, and *causally consistent*, in the sense of preserving the order and causal relations between events. Modern operating systems expose this stream through their kernel auditing subsystems. On Linux, the `auditd` framework and its more recent extension based on eBPF tracing provide hooks at the level of system calls. On Windows, the Event Tracing for Windows (ETW) facility plays an analogous role.

Several research systems have been built on top of these primitives. SPADE (Gehani and Tariq, 2012) offers a portable framework for capturing and querying provenance across multiple operating systems. CamFlow (Pasquier et al., 2017) adopts a tighter integration with the Linux kernel via Linux Security Modules. CamFlow provides whole-system provenance with low runtime

overhead. The DARPA Transparent Computing programme sponsored the development of dedicated sensors. These sensors, including the TRACE sensor used throughout this thesis, output a normalized event stream conforming to a common data model known as CDM18.

The construction pipeline shared by all these systems can be summarized in three stages. In the first stage, kernel hooks intercept each system call of interest and emit a structured record. In the second stage, a normalization layer maps every record to a uniform schema. This schema describes the source entity, the target entity, the operation type, and the timestamp. In the third stage, an aggregation layer turns the resulting event stream into a graph. It does so by maintaining an evolving set of nodes, one per observed entity, and by inserting an edge whenever a new event is recorded. The dataset used throughout this thesis, DARPA TRACE, is the output of precisely such a pipeline applied to a controlled adversarial exercise. Its schema is described in detail in Chapter 4.

1.3.3 Granularity, scale, and reduction techniques

The expressiveness of provenance graphs comes at a substantial cost: their scale. A single workstation under normal use can generate hundreds of millions of events per day. An enterprise-scale deployment trivially crosses the billion-event mark. The resulting graphs do not fit on commodity hardware, and naively analyzing them in real time is computationally prohibitive.

A second difficulty compounds the issue. It is often referred to as the dependency-explosion problem. A long-lived process such as a shell or a browser interacts with thousands of files and network endpoints over its lifetime. When forensic queries are issued against the provenance graph: *what did this process touch, what touched this file*, the answer rapidly expands into a hairball that obscures the relevant causal paths. Reducing this hairball without losing the information needed for detection is an active research direction (Hossain et al., 2020; Kwon et al., 2018).

Three families of reduction techniques have emerged. *Causal trimming* removes edges and nodes that cannot have contributed to a suspicious event. It does so by following dependency relations backwards from a seed node. *Semantic compression* merges sequences of equivalent operations into a single edge. It exploits the fact that many bursts of identical system calls have the same causal meaning (Yang et al., 2023). *Sampling* selects a subset of edges or neighbours for analysis. It trades completeness for tractability.

Sampling is particularly relevant to the present work. Graph neural networks operating on provenance graphs cannot, in general, load the full neighborhood of every node into GPU memory. Neighborhood sampling: selecting at random a bounded number of neighbors per node at each layer is the standard solution. The regime under which this sampling is performed has, as our experiments will show, non-trivial consequences on the detection performance of the resulting model.

1.4 Graph Learning for Security

1.4.1 Message passing on heterogeneous graphs

Graph neural networks (GNNs) are a family of deep-learning architectures designed to operate on graph-structured inputs (Bilot et al., 2023). Their unifying idea is *message passing*. At each layer, every node updates its representation by aggregating information received from its neighbors. It then combines this aggregated information with its own previous representation. After L layers of message passing, each node embeds information about its L-hop neighborhood. Downstream classifiers can then reason about structural context rather than about isolated entities.

Within this framework, GNN variants differ along three axes. The first axis is the *aggregation function* used to combine messages from neighbors, such as sum, mean, attention-weighted, or max. The second axis is the *combination function* used to merge the aggregated message with the node's previous state. The third axis is the *layer normalization and non-linearity* used to stabilize training. The graph convolutional network, GraphSAGE (Hamilton et al., 2017), and the graph attention network (Veličković et al., 2018) are canonical instances of this template. Each of them is examined in detail in Chapter 4.

Provenance graphs introduce an additional layer of complexity: *heterogeneity*. Their nodes and edges carry types. A process is not a file, and an exec is not a read. These types convey semantic information that a type-agnostic aggregation would discard. GNN designs targeted at heterogeneous graphs explicitly condition the aggregation function on edge and node types. They apply type-specific transformations before combining messages. The STGAN model, studied in this thesis, embeds the node type as a categorical feature and lets the attention mechanism learn to weight messages accordingly. This approach strikes a pragmatic balance between expressiveness and computational cost.

1.4.2 Embedding-based representations of system entities

A graph neural network requires a numerical representation of each node before message passing can begin. For provenance graphs, this requirement is non-trivial. The entities are identified by strings such as process command lines, file paths, and network endpoints. These strings are drawn from an effectively unbounded vocabulary. Encoding them into fixed-dimensional vectors is the role of an *embedding model*.

The earliest provenance-based detectors used hand-crafted features. These features included counts of system-call types, statistics of subgraph motifs, and the presence of suspicious tokens. The features were interpretable but brittle. Small changes in the attacker's tooling rapidly invalidated them. Subsequent work adopted learned representations drawing on the distributional

hypothesis from natural language processing. According to this hypothesis, entities that appear in similar contexts should be embedded close to each other. Static word embeddings such as Word2Vec, FastText, and node2vec (Grover and Leskovec, 2016) produce such vectors. They are trained on a corpus of token sequences extracted from the graph (Narayanan et al., 2017).

More recent approaches use *contextual* embeddings derived from pre-trained transformer language models. Rather than assigning each token a fixed vector, these models produce a representation that depends on the surrounding context. The representation can therefore capture polysemy and longer-range syntactic structure. As is discussed in Chapter 3, the choice between static and contextual embeddings, and between embeddings of different families within each category, has been sparsely studied in the provenance-graph setting. The first contribution of this thesis fills this gap by comparing four embedding families within a single, controlled experimental protocol.

1.5 Conclusion

This chapter has established the conceptual framework on which the rest of the thesis builds. The starting point is the observation that modern endpoint threats, and Advanced Persistent Threats in particular, manipulate sequences of operations whose maliciousness is not detectable on a per-event basis. Provenance graphs represent these sequences as causal and temporal relationships between system entities. They thus provide the substrate over which sophisticated, multi-stage attacks can be reasoned about holistically.

Operating on these graphs requires two complementary tools. The first tool is an embedding model that encodes the identity and the semantics of each entity into a fixed-dimensional vector. The second tool is a graph neural network that aggregates information across causal neighborhoods. These two design choices, which embedding family to use and which graph neural network architecture to use, are precisely the levers explored by the first two contributions of this thesis.

The next chapter situates these choices within the specific context of the project. It introduces the baseline model on which we build, namely STGAN, and it formulates the research questions that motivate the comparative studies reported in the subsequent chapters.

Chapter 2

Context and Problem Statement

2.1 Introduction

The previous chapter established the general framework of provenance-graph-based intrusion detection. This chapter situates the work within this framework. It introduces the baseline model on which the project builds, identifies the open questions that motivate the work, and states the concrete objectives that the rest of the thesis pursues.

2.2 The STGAN Model: Promise and Open Questions

2.2.1 Why STGAN was selected as the baseline

The baseline of this project is **STGAN** (Sang et al., 2025), a learning-based provenance-graph detector published at The ACM Web Conference 2025 (WWW '25). STGAN was retained over alternative systems for three converging reasons.

First, STGAN is the first published system to fuse *three* complementary streams of information: structural, semantic, and temporal within a single end-to-end architecture. Earlier detectors typically capture one or two of these dimensions in isolation. The temporal-only family of dynamic graph learners overlooks the rich semantic identity of system entities. The structural-only family of static graph learners overlooks the time order of events. STGAN is therefore representative of the most recent paradigm in the field.

Second, the reported detection performance of STGAN is, at the time of writing, among the highest published on the DARPA Transparent Computing Engagement 3 benchmark. The reported F1 score on the TRACE subset is 0.9916, surpassing a panel of contemporary detectors including HOLMES(Milajerdi et al., 2019), Unicorn(Han et al., 2020), Threatrace(Wang et al., 2022), and FLASH(Ur Rehman et al., 2024). Working with a strong baseline is essential for any ablation effort. The conclusions drawn from comparing alternative designs are meaningful only

when the reference design is itself state-of-the-art.

Third, the source code and the pre-trained weights of STGAN are released under an open licence. The availability of these artefacts is a prerequisite for the kind of empirical reproduction and design-substitution study that this project conducts. The released artefacts also provide an objective ground truth against which our own reproduction can be benchmarked, as discussed in Chapter 6.

2.2.2 The role of the semantic stream

The semantic stream of STGAN encodes the identity and the lexical content of each system entity. The encoding starts from a textual representation of the entity. For a process, this representation includes its executable path and command-line arguments. For a file, it is the canonical path. For a network endpoint, it is the protocol-address-port triple. STGAN tokenises these strings into bounded sequences of words and treats each neighborhood-derived sequence as a sentence drawn from a corpus of legitimate system activity.

The semantic encoder is a Word2Vec model trained on this corpus. It produces a fixed-dimensional embedding vector for each token, and the per-entity embedding is then obtained by averaging the embeddings of the entity's tokens. This choice is justified in the original publication on grounds of efficiency and of the empirical observation that benign and malicious entities tend to use distinct lexical fields.

The choice of Word2Vec is, however, only one of several plausible options. Several embedding families have been developed since Word2Vec. Each family offers different inductive biases. Subword embeddings such as FastText handle out-of-vocabulary tokens by composition. Contextual transformer embeddings such as BERT and EmbeddingGemma capture polysemy and longer-range token dependencies. The published ablation of STGAN does not compare Word2Vec to any of these alternatives. The first contribution of this thesis is therefore to fill this gap.

2.2.3 The role of the structural and fusion components

The structural component of STGAN is a graph attention network (Veličković et al., 2018). It aggregates information across causally connected entities by computing learned attention coefficients between each node and its neighbors. The attention mechanism allows the model to weight different neighbors differently. This property is useful on provenance graphs, where the type and the timing of an edge carry semantic significance.

The fusion component of STGAN is a multi-head self-attention module. It combines the spatial and temporal features produced by the upstream encoders into a single representation. Each attention head learns a distinct linear projection of the input features and computes a separate

attention pattern. The outputs of all heads are then concatenated and projected.

Both of these components are attention-based, and both incur a non-trivial computational cost. The published ablation of STGAN compares the graph attention network to a graph convolutional network and the multi-head fusion to a single-head self-attention. The comparison does not, however, extend to neighborhood-aggregation backbones such as GraphSAGE (Hamilton et al., 2017) or to non-attention fusion modules such as pooling and averaging. The second contribution of this thesis is to extend the comparison along these axes. Its goal is to identify whether the attention-based choices made by STGAN can be replaced by simpler alternatives without sacrificing detection accuracy.

2.3 Problem Statement

2.3.1 The impact of the embedding choice

The semantic stream of STGAN relies on Word2Vec to encode each system entity. Word2Vec is a static word-level embedding model trained on the corpus of entity tokens. It assigns a fixed vector to each known token and computes the embedding of an entity as the mean of its token vectors. The choice predates the rise of subword and contextual representations by nearly a decade. Several embedding families have since become standard in adjacent fields of natural language processing and graph learning. None of them has been evaluated within the STGAN architecture in the original publication.

The first research question of this thesis is therefore the following. *Does the substitution of Word2Vec by a more recent embedding family yield a measurable change in detection performance, when all other components of STGAN are held constant?* Three alternatives are considered. The first is FastText, a subword embedding model that handles out-of-vocabulary tokens by composing them from learned character n-grams. The second is BERT, a contextual transformer-based embedding model that produces token representations dependent on the surrounding context. The third is EmbeddingGemma, a more recent transformer embedding model designed to be steerable through task-aware prompts. The answer to this question has direct consequences for ProvSAGE: a positive answer indicates that a richer embedding contributes to detection accuracy, while a negative answer indicates that Word2Vec is sufficient and that computational complexity should be invested elsewhere, such as in the structural-temporal fusion layers.

2.3.2 The contribution of attention-based components

The structural and fusion modules of STGAN are both attention-based. The structural module is a graph attention network, which computes learned attention coefficients between each node and its neighbors. The fusion module is a multi-head self-attention layer, which weights the

contribution of the spatial and temporal features. Both modules carry a non-negligible computational cost. The parameter count is dominated by the multi-head fusion, and the training time per epoch is dominated by the attention coefficient computations of the structural module.

The second research question of this thesis is therefore the following. *Can the attention-based modules of STGAN be replaced by simpler aggregation primitives without a significant loss in detection performance?* For the structural module, two non-attention alternatives are considered. The first is GraphSAGE, which aggregates a sampled subset of each node's neighbours by learned mean or pooling operators. The second is the graph convolutional network, which aggregates the full neighborhood with fixed normalization coefficients. For the fusion module, two non-attention alternatives are considered. The first is max-pooling across feature dimensions. The second is element-wise averaging. The answer to this question identifies precisely which parts of STGAN can be safely lightened and which must be preserved.

2.3.3 The structure of STGAN's anomaly set for investigation

STGAN, like most node-level detectors, returns a set of entities flagged as anomalous. For real-world deployment, the flag set is rarely the final output of interest. An analyst typically needs to understand how the flagged entities relate to one another, which entity initiated the intrusion, and which downstream entities were ultimately affected. The natural complement to a node-level detector is therefore a post-detection investigation module that organises the flag set into an interpretable attack narrative.

The third research question of this thesis concerns the design of such a module under a strict deployability constraint. The question is formulated as follows. *Can the entities flagged by STGAN be compressed into a compact subgraph representing the candidate attack neighborhood, using only signals that are available at inference time?* The deployability constraint excludes any access to the ground-truth label set, in contrast to the evaluation-time helpers used in the published STGAN ablation. The intent of the question is twofold. It first seeks a positive algorithmic answer, in the form of a reproducible module with a measurable compression-versus-precision trade-off. It then seeks to characterise the structural property of STGAN's flag set that determines whether such a compression is feasible at all.

2.4 Project Objectives

2.4.1 Reproduction of the baseline

The first objective of the project is to reproduce STGAN end-to-end on the DARPA TRACE dataset under conditions that we control. The reproduction serves three purposes. The first is to verify that the public release of STGAN, including its source code, its pre-trained weights, and its data preparation pipeline, is sufficient for an independent practitioner to obtain the published detection performance. The second is to expose any reproducibility gap between the publication and the available artefacts. The third is to provide a deterministic reference run against which the alternative configurations of the following objectives can be compared.

Two complementary reproduction experiments are conducted. The first experiment loads the publicly released STGAN weights and runs them on our local copy of the TRACE test set. The second experiment retrains STGAN from scratch using the same code, but with deterministic settings (a fixed random seed and deterministic CUDA kernels). The two experiments together provide a complete picture of the reproducibility of the published baseline.

2.4.2 Comparative study of semantic embeddings

The second objective of the project is to answer the first research question by an empirical comparison of semantic embedding families within the STGAN architecture. The Word2Vec module of STGAN is replaced by four alternative embedding models, one at a time. The structural module, the temporal module, and the fusion module are kept strictly identical across all configurations. The data, the training procedure, and the evaluation protocol are also held constant. Any difference in detection performance can therefore be attributed exclusively to the embedding choice.

The four alternative embeddings are FastText, BERT, EmbeddingGemma in its raw form, and EmbeddingGemma in a prompted form. FastText is selected as a representative subword embedding. BERT is selected as a representative contextual transformer embedding of moderate size. EmbeddingGemma is selected as a more recent transformer embedding that can be steered by a task-aware prompt; the raw and prompted variants are compared to isolate the effect of the prompt itself. The five configurations (Word2Vec baseline plus the four alternatives) constitute the first contribution of this thesis.

2.4.3 Architectural ablation

The third objective of the project is to answer the second research question by an empirical comparison of architectural alternatives within the STGAN template. The comparison proceeds along two independent axes. Along the first axis, the structural module is varied: the graph

attention network of STGAN is replaced by GraphSAGE and by a GCN. Along the second axis, the fusion module is varied: the multi-head self-attention of STGAN is replaced by max-pooling and by element-wise averaging. The semantic stream, the temporal stream, and the data preparation pipeline are kept identical across all configurations.

Five configurations are evaluated in total. The first configuration is the baseline (graph attention network + multi-head fusion). Two configurations vary the structural axis (GraphSAGE + multi-head fusion and graph convolutional network + multi-head fusion). The remaining two configurations vary the fusion axis (graph attention network + pooling and graph attention network + averaging). The results of this comparative study identify the recommended ProvSAGE configuration and constitute the second contribution of this thesis.

2.4.4 Investigation study

The fourth objective of the project is to answer the third research question by designing and validating a post-detection investigation module. The module is named ProvSAGE-C3 and operates as a four-stage pipeline on the raw flag set returned by STGAN. The first stage filters the flag set by entity type, retaining only the two CDM18 types empirically associated with anonymous inter-process communication and character-device I/O (FILE_OBJECT_CHAR, type 2; UnnamedPipeObject, type 6). These two types match the entry points through which the red-team scenarios of the DARPA TRACE engagement exfiltrate data via stdout/stderr redirections and anonymous pipes, as documented in the host-level intrusion forensics literature. The second stage identifies dense temporal windows in the activity of the filtered anchors, using a sliding-window analysis on the median timestamp of each anchor's edges. The third stage produces, for each of the top dense windows, a one-hop topological expansion of the anchors that fall within the window. The fourth stage ranks the resulting candidate subgraphs by their topological compactness and selects the most compact candidate as the predicted attack neighborhood.

The validation of the module is conducted on the DARPA TRACE test set. The output candidate subgraph is compared to the ground-truth list of malicious entities to measure the compression ratio, the recall of correctly-flagged anchors retained in the candidate, and the count of additional confirmed-malicious entities recovered by the topological expansion. The ground truth is used only at this validation stage. The module itself relies exclusively on signals available at inference time: entity type, edge timestamps, and provenance-graph topology. This third contribution provides the practical investigation aid that the cybersecurity domain charter requires for the public defence.

2.4.5 Reproducibility and practical demonstration

The fifth objective of the project is to ensure that every result reported in the experimental chapter can be independently reproduced.

The reproducibility artefacts include the source code of every training and evaluation script, the trained weights of every configuration reported in the experimental chapter, the deterministic random seeds used at every step of the pipeline, and a set of Jupyter notebooks that orchestrate the end-to-end workflow. The notebooks are designed so that a reader equipped with a single A100 GPU and the relevant dataset access can re-execute every experiment from raw audit logs to the final detection metrics.

2.5 Conclusion

This chapter has framed the project. STGAN was identified as the appropriate baseline on which to build. Three open questions were extracted from a careful reading of the STGAN publication. These questions concern the impact of the semantic embedding, the contribution of the attention-based modules, and the design of a deployable post-detection investigation module. Five concrete objectives were derived from these questions. They cover the reproduction of the baseline, two comparative studies, the design and validation of an investigation module, and the reproducibility artefacts that support the practical demonstration.

The next chapter surveys the state of the art on which the project draws. It presents the lineage of learning-based detectors on provenance graphs and the families of semantic embeddings considered in the comparative study.

Chapter 3

State of the Art: Learning on Provenance Graphs

3.1 Introduction

This chapter situates ProvSAGE in the wider landscape of learning-based intrusion detection on provenance graphs. The presentation focuses on the families of systems and the families of techniques that are directly relevant to the contributions of this work: provenance-based detection systems, semantic representations of system entities, and lightweight graph neural networks. The chapter ends with an explicit statement of the gaps that the contributions of this work address.

3.2 Detection Systems Based on Provenance Graphs

The provenance-graph paradigm has produced, over the past five years, a sustained line of learning-based detection systems. The systems differ along three principal axes: the structural snapshot they consume, the temporal granularity at which they operate, and the position of the semantic stream within the architecture. The following subsections survey these axes and locate STGAN within them.

3.2.1 Static and snapshot-based approaches

The earliest learning-based detectors operate on a single static snapshot of the provenance graph. UNICORN (Han et al., 2020) pioneered this family by maintaining an incrementally updated Weisfeiler-Lehman sketch of the host graph and detecting deviations from a learned benign baseline through histogram distances. ProGrapher (Yang et al., 2023) extends the static paradigm by applying graph2vec to fixed-window snapshots and consuming the resulting graph embeddings with a TextRCNN classifier. ThreaTrace (Wang et al., 2022) produces node-level decisions on

the static snapshot by training a multi-model GraphSAGE classifier whose confidence is used as the anomaly score. FLASH (Ur Rehman et al., 2024) combines a Word2Vec representation of node-level token sequences with a GraphSAGE-based aggregator and produces compact Attack Evolution Graphs from the node-level decisions.

The common limitation of the static family is that the snapshot abstracts away the time dimension. The decisions produced by these systems reflect the structural state at the moment of the snapshot, but cannot in general distinguish a slow-moving intrusion from a bursty administrative activity that produces a structurally similar snapshot. The temporal axis must therefore be reintroduced explicitly, either by snapshot sequencing or by continuous-time modelling.

3.2.2 Temporal and dynamic approaches

The temporal family of detectors operates on the sequence of events rather than on a single snapshot. CGL-AD (Wang et al., 2025) segments the event stream into fixed-length windows, computes a TGN-based embedding per window, and consumes the resulting embedding sequence with a bidirectional recurrent network. Pikachu (Paudel and Huang, 2022) applies a temporal walk to the dynamic graph and learns embeddings from the time-ordered walk sequences. Euler (King and Huang, 2023) combines a per-snapshot GNN encoder with a GRU temporal module and detects lateral movement as the edges whose predicted probability falls below a threshold. The work of Reha et al. (Reha et al., 2023) formalizes the streaming case with a heterogeneous temporal graph network that updates per-node memory states at every event.

Within this family, STGAN (Sang et al., 2025) occupies a hybrid position. The system processes the provenance graph in fixed-window batches, like the snapshot family, but applies a temporal graph network within each window to capture the within-window dynamics. The output of the temporal stream is then fused with the structural and semantic streams by a multi-head self-attention mechanism. This hybrid is the design that ProvSAGE inherits and that the contributions of this work refine.

3.2.3 Position of STGAN in this landscape

STGAN distinguishes itself from the systems just surveyed by its multi-stream fusion architecture. Most prior detectors operate on a single representational axis (structural, semantic, or temporal); STGAN is the first system to combine the three within a single trainable architecture and to report state-of-the-art performance on the DARPA TRACE benchmark. The system relies on Word2Vec for the semantic axis, the graph attention network of Veličković et al. (Veličković et al., 2018) for the structural axis, and the temporal graph network of Rossi et al. (Rossi et al., 2020) for the temporal axis. The multi-head fusion combines the three.

This integration is the strength of STGAN, but also the source of the open questions addressed by this thesis. The published ablation study of STGAN reports that each of the three streams

contributes to detection performance but does not extend the comparison to alternative implementations within each stream. The remaining sections of this chapter survey the literature on the families of alternatives that ProvSAGE considers.

Table 3.1: Position of representative provenance-based detection systems along the three axes used to organize the chapter.

System	Structural	Semantic	Temporal	Embedding
UNICORN (Han et al., 2020)	sketch	–	snapshot	–
ProGrapher (Yang et al., 2023)	graph2vec	–	snapshot	graph2vec
ThreaTrace (Wang et al., 2022)	GraphSAGE	node tokens	snapshot	one-hot
FLASH (Ur Rehman et al., 2024)	GraphSAGE	Word2Vec	snapshot	Word2Vec
Pikachu (Paudel and Huang, 2022)	rand. walk	–	continuous	temporal walk
CGL-AD (Wang et al., 2025)	TGN	–	window-based	–
Euler (King and Huang, 2023)	GNN+GRU	–	window-based	–
STGAN (Sang et al., 2025)	GAT	Word2Vec	TGN + MHA	Word2Vec
ProvSAGE (this work)	GraphSAGE	W2V/FT/BERT/Gemma	TGN + MHA	W2V (best)

3.3 Semantic Representations of System Entities

The semantic stream of a provenance-based detector maps each system entity to a vector that captures its identity and context. The choice of representation conditions both the detector's expressiveness and its computational cost. This section surveys the families of representations that have been applied to provenance data.

3.3.1 From bag-of-words to learned embeddings

The earliest semantic representations of system entities were bag-of-words encodings of the entity name and its type. These encodings produce sparse, high-dimensional vectors whose dimensionality grows with the vocabulary and whose semantic content is limited to the identity of the tokens that appear in the name. The transition to learned embeddings, in which each entity is represented by a dense vector in a fixed-dimensional space, was motivated by the need to capture semantic similarity between entities whose names share no surface tokens.

3.3.2 Static word embeddings

The static-embedding family produces a single vector per token, independently of the context in which the token appears. Word2Vec (Mikolov et al., 2013) is the canonical representative of this family and is the embedding used by FLASH (Ur Rehman et al., 2024) and by STGAN (Sang et al., 2025). FastText (Bojanowski et al., 2017) extends Word2Vec by representing each token as the sum of its character-n-gram embeddings, which gives the model the ability to produce

embeddings for tokens unseen at training time. The character-n-gram decomposition is well aligned with the structure of system-entity names, which contain frequent suffixes such as `.so`, `.log`, and `tmp`. The first contribution of this thesis evaluates whether this alignment translates into a measurable detection gain on DARPA TRACE.

3.3.3 Contextual transformer embeddings

The contextual-embedding family produces a vector per token that depends on the surrounding context. The pre-trained sentence encoders of the BERT family (Devlin et al., 2019; Reimers and Gurevych, 2019) and the open-weight decoder of Gemma (Gemma Team, Google DeepMind, 2024) are the two representatives evaluated in this work. The expressive power of these models is well-documented on natural-language benchmarks, but their alignment with the short, system-oriented strings that constitute provenance-entity names is less well understood. The first contribution of this thesis treats this alignment as an empirical question and reports the corresponding results in Chapter 6

3.4 Lightweight Graph Neural Networks

The structural stream of a provenance-based detector aggregates information from the neighborhood of each node. The choice of aggregator conditions both the expressiveness of the structural representation and the computational cost of the detector.

3.4.1 Trade-offs between attention and aggregation

The graph attention network of Veličković et al. (Veličković et al., 2018) computes per-edge attention coefficients and aggregates neighbor information by an attention-weighted sum. The attention mechanism allows the network to weight neighbors differentially but introduces a per-edge multi-head computation whose cost dominates the structural stream of STGAN. The graph convolutional network of Kipf and Welling (Kipf and Welling, 2017) and the GraphSAGE inductive aggregator of Hamilton et al. (Hamilton et al., 2017) replace the per-edge attention by a fixed, parameter-free aggregation. The two aggregators are an order of magnitude lighter than GAT and have been applied successfully to large-scale node-classification benchmarks.

3.4.2 Position of GraphSAGE and GCN

GraphSAGE and GCN have already been applied to provenance-based detection. ThreaTrace (Wang et al., 2022) and FLASH (Ur Rehman et al., 2024) both rely on GraphSAGE as the structural aggregator and report state-of-the-art performance on the DARPA benchmarks. No prior work has substituted GraphSAGE or GCN for the GAT layer of STGAN. The second contribution of this thesis fills this gap with a complete ablation of the structural and the fusion modules.

3.5 Remaining Gaps Addressed in This Work

The literature survey identifies three gaps that this thesis addresses. The first gap concerns the semantic stream: no prior work has measured the impact of replacing the Word2Vec embedding of STGAN by FastText, by BERT, or by Gemma. The second gap concerns the structural and fusion modules: no prior work has measured the impact of replacing the GAT layer of STGAN by GraphSAGE or by GCN, nor the impact of replacing the multi-head attention fusion by mean pooling or simple averaging. The third gap concerns the investigative use of STGAN: the system reports per-anchor decisions but does not produce an attack neighborhood that supports forensic interpretation. The three contributions of this thesis address these three gaps in order.

3.6 Conclusion

This chapter has situated ProvSAGE in the literature on provenance-based intrusion detection. The survey has identified three concrete gaps in the published evaluation of STGAN and has motivated the three contributions of this thesis. The next chapter presents the technical foundations on which these contributions rely.

Chapter 4

Technical Foundations: STGAN, Dataset and Embedding Models

4.1 Introduction

This chapter presents the technical building blocks on which the contributions of this thesis rely. The chapter is organized around the three streams of the STGAN architecture (semantic, structural, temporal) and the fusion module that combines them. Each building block is presented at the level of detail required to understand the experimental chapters; the reader is referred to the cited literature for the full mathematical treatment.

4.2 The STGAN Architecture

STGAN (Sang et al., 2025) is a multi-stream graph neural network for provenance-based host intrusion detection. The architecture combines three representational streams (semantic, structural, temporal) with a multi-head self-attention fusion module and a self-distillation training procedure. The following subsections describe each of the five components in turn.

4.2.1 Overview: three streams and a fusion

The architecture takes as input a window-based batch of the provenance event stream. Each event is consumed in parallel by three streams. The semantic stream embeds the name of each entity into a dense vector. The structural stream aggregates the embeddings of the neighbors of each entity by a graph attention layer. The temporal stream updates a per-node memory state by a temporal graph network. The three stream outputs of each node are then combined by a multi-head self-attention layer into a single fused embedding. A classification head consumes the fused embedding and produces a per-node anomaly score. The overall architecture is summarized in Figure 4.1.

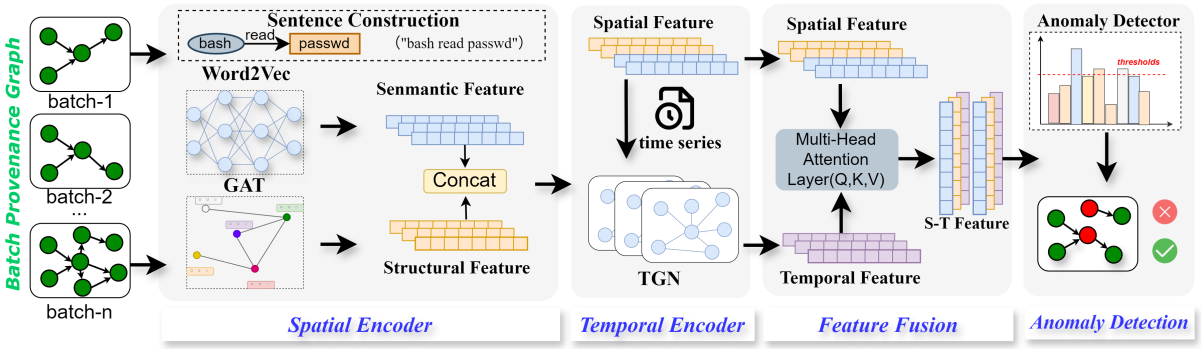


Figure 4.1: Architecture of STGAN, the baseline on which ProvSAGE builds. Adapted from Sang et al. (Sang et al., 2025).

4.2.2 Structural stream: Graph attention network

The structural stream is a two-layer multi-head graph attention network (Veličković et al., 2018). Each layer computes, for every edge, an attention coefficient derived from the embeddings of the two endpoints, and aggregates the neighbors of each node by the corresponding attention-weighted sum. The multi-head variant computes several attention maps in parallel and concatenates the resulting embeddings. The output of the second layer is the structural embedding of each node. The structural stream dominates the computational cost of the architecture on dense provenance graphs.

4.2.3 Semantic stream: Word2Vec and positional encoder

The semantic stream maps the name of each entity to a dense vector via a Skip-gram Word2Vec model (Mikolov et al., 2013) trained on the corpus of all entity names of the training graph. A positional encoder of the form proposed by Vaswani et al. (Vaswani et al., 2017) is added to the Word2Vec embedding to inject a notion of position into the otherwise order-invariant representation. The combined output is the semantic embedding of each node.

4.2.4 Temporal stream: Temporal graph network

The temporal stream is an instance of the temporal graph network of Rossi et al. (Rossi et al., 2020). The module maintains a per-node memory state that is updated at every event. The update is performed by a learned message function whose inputs are the previous memory states of the two endpoints, the embedding of the event, and the inter-event time difference. The memory state is read by an embedding module that combines it with the structural embedding of the node and produces the temporal embedding. The temporal stream is the only component that introduces an explicit time dimension into the architecture.

4.2.5 Fusion and classification: multi-head self-attention

The fusion module treats the three stream outputs of each node as a three-token sequence and applies a multi-head self-attention layer to it (Vaswani et al., 2017). The output is a fused embedding of the same dimensionality as each stream output. A two-layer multilayer perceptron consumes the fused embedding and produces the per-node anomaly score. The score is thresholded at training time to yield a binary classification.

4.2.6 Training procedure and self-distillation mask

The architecture is trained end-to-end by stochastic gradient descent on the binary cross-entropy of the per-node anomaly score against the ground-truth label. A self-distillation mask is applied during training: the score of a fraction of the malicious nodes is masked at random, and the model is forced to recover the masked decision from the remaining context. The mask acts as a form of regularization and improves the generalisation of the classifier to unseen malicious patterns.

4.3 The DARPA TRACE Dataset

The contributions of this thesis are evaluated on the DARPA TRACE dataset. This section presents the origin of the dataset, the schema in which the events are recorded, and the train/test split adopted in this work.

4.3.1 Origin and motivation

DARPA TRACE was released as part of the Transparent Computing programme of the United States Defense Advanced Research Projects Agency. The programme aimed at producing a publicly available benchmark for host-level intrusion detection on operating-system provenance data. The TRACE engagement collected several days of system-call activity on instrumented FreeBSD hosts, during which red-team operators executed scripted intrusion scenarios. The collected stream contains both benign administrative activity and the malicious activity of the red-team scenarios. The dataset has become the de facto benchmark for evaluating provenance-based detection systems and is the dataset used in the published evaluation of STGAN.

4.3.2 The CDM18 schema and the raw JSON format

The events of DARPA TRACE are serialized in the Common Data Model version 18 (CDM18) format. Each event is a JSON record whose schema is the standard published by the Transparent Computing programme. The schema defines twelve entity types (six `FILE_OBJECT_*` subtypes, `SUBJECT_PROCESS`, `SUBJECT_UNIT`, `MemoryObject`, `NetFlowObject`, `UnnamedPipeObject`,

SRCSINK_UNKNOWN) and the corresponding edge types. Each event carries a UUID for each endpoint, a system-call type, and a microsecond-resolution timestamp. The raw stream is pre-processed by the STGAN pipeline into a sequence of typed, time-stamped edges over the twelve entity types.

4.3.3 Train/test split, ground truth, and statistics

The train/test split of this work follows the published split of STGAN. The training set contains the events of the benign-only segment of the engagement; the test set contains the events of the segment during which the red-team scenarios were executed. The ground-truth label of each entity is derived from the red-team after-action report and is published alongside the dataset. The training set contains approximately 2.7 million entities and 26.5 million events; the test set contains approximately 1.21 million nodes of which 1 248 are labelled malicious by the after-action report.

4.4 Semantic Embedding Families

The first contribution of this thesis evaluates four families of semantic embeddings as substitutes for the Word2Vec stream of STGAN. The four embeddings are presented in turn.

4.4.1 Word2Vec: static word-level embedding

Word2Vec (Mikolov et al., 2013) learns one vector per token from the co-occurrence statistics of a training corpus. The Skip-gram variant used by STGAN trains the embedding by predicting the context tokens given the centre token, with negative sampling as the optimization objective. The output is a static lookup table from token to vector. The embedding is fast to train, has no inference cost beyond the table lookup, and produces a 30-dimensional vector per token in the STGAN configuration. Tokens unseen at training time receive no embedding and are handled by an out-of-vocabulary policy that maps them to the zero vector.

4.4.2 FastText: static sub-word embedding

FastText (Bojanowski et al., 2017) represents each token as the sum of the embeddings of its character n-grams. The character-n-gram decomposition has two consequences. First, the model can produce an embedding for any token, including tokens unseen at training time, by summing the embeddings of its character n-grams. Second, the model captures sub-word morphology: two tokens that share a frequent suffix or prefix share the corresponding n-gram components of their embedding. Both properties align well with the structure of system-entity names, which contain frequent suffixes such as `.so`, `.log`, and `tmp`. The cost of FastText at inference time is the cost of one table lookup per character n-gram.

4.4.3 BERT: contextual transformer embedding

BERT (Devlin et al., 2019) is a transformer encoder pre-trained on a large corpus by masked language modelling and next-sentence prediction. The contextual nature of the embedding is realized by the self-attention mechanism, which computes for each token a weighted sum of the embeddings of the other tokens of the sequence. The variant used in this work is the Sentence-BERT bi-encoder of Reimers and Gurevych (Reimers and Gurevych, 2019), which produces a 384-dimensional vector per input string by mean pooling the last hidden states. The embedding is reduced to the 30-dimensional space expected by the STGAN pipeline by a learned linear projection trained end-to-end with the downstream classifier.

4.4.4 EmbeddingGemma: task-aware contextual embedding

EmbeddingGemma (Gemma Team, Google DeepMind, 2024) is the open-weight embedding model of the Gemma family, released by Google DeepMind. The model is a decoder-style transformer of 308 million parameters that produces a 768-dimensional vector per input string. The model accepts an optional task-specific instruction prepended to the input, which conditions the embedding on the downstream task. Both the unprompted and the task-prompted variants are evaluated in Chapter 6. As for BERT, the output is reduced to the 30-dimensional space expected by the STGAN pipeline by a learned linear projection.

4.5 Lightweight Graph Neural Networks Considered

The second contribution of this thesis evaluates two lightweight graph neural networks as substitutes for the graph attention layer of STGAN. The two aggregators are presented in turn.

4.5.1 GraphSAGE: aggregation without attention

GraphSAGE (Hamilton et al., 2017) samples a fixed number of neighbors per node and aggregates the sampled neighbors by a parameter-free function. The mean aggregator, used in this work, averages the embeddings of the sampled neighbors and concatenates the result with the previous embedding of the node. A learned linear layer projects the concatenation into the next-layer space. The cost of the structural stream is reduced to a quantity independent of the maximum degree of the graph, and the per-edge attention computation of GAT is eliminated.

4.5.2 GCN: graph convolution

The graph convolutional network of Kipf and Welling (Kipf and Welling, 2017) aggregates neighbors by a normalized sum. The normalization coefficient of an edge is the inverse of the square root of the product of the degrees of the two endpoints. The aggregation is parameter-

free; the only parameters of the layer are the weights of the linear message function applied to the embeddings of the neighbors. GCN is the lightest of the three aggregators considered in this work but does not differentiate between neighbors, which may be a limitation on heterogeneous provenance graphs.

4.6 Conclusion

This chapter has presented the technical building blocks on which the contributions of this thesis rely: the STGAN architecture, the DARPA TRACE dataset, the four semantic embedding families evaluated in Contribution 1, and the two lightweight graph neural networks evaluated in Contribution 2. The next chapter presents the design and the implementation of ProvSAGE.

Chapter 5

ProvSAGE: Design and Methodology

5.1 Introduction

This chapter describes the design and the methodology of ProvSAGE, the variant of STGAN proposed in this thesis. ProvSAGE retains the overall three-stream architecture of STGAN and the helper-based evaluation pipeline of the original publication, but extends the published model with three contributions. The first contribution is a comparative study of the semantic embedding module. The second contribution is an ablation of the structural and fusion modules. The third contribution is a post-detection investigation module that returns a compact attack-neighborhood subgraph. Each contribution is preceded by a controlled reproduction of the published baseline so that the comparative findings can be attributed to the design variations under study.

The chapter is organized as follows. Section 5.1 describes the reproduction of the STGAN baseline and the experimental conditions held constant across the rest of the work. Section 5.2 describes the comparative embedding study. Section 5.3 describes the architectural ablation. Section 5.4 describes the investigation module. Section 5.5 collects the implementation notes shared across the three contributions.

5.2 Reproduction of the STGAN Baseline

The reproduction of the STGAN baseline is the prerequisite to every comparative study reported in this thesis. The reproduction has two purposes. The first purpose is to verify that the publicly released artefacts of STGAN are sufficient for an independent practitioner to obtain a detection performance comparable to the one reported in the original paper. The second purpose is to provide a reference run against which the alternative configurations of the next sections are compared, under conditions held strictly constant.

5.2.1 Hardware and software environment

All experiments are conducted on an A100 80 GB graphics processing unit configured as a MIG slice of 20 GB of dedicated memory, hosted on a Linux workstation with 251 GB of system memory. The software stack is based on Python 3.11, PyTorch 2.6.0 with CUDA 12.4, and PyTorch Geometric 2.7.0. The semantic embedding training relies on gensim 4.4.0 for Word2Vec and FastText, and on Hugging Face transformers and sentence-transformers for BERT and EmbeddingGemma. The XGBoost-based downstream detector of the original STGAN implementation is replaced by the cross-entropy loss provided by the public code release, which is the version effectively executed by the released training scripts.

5.2.2 Data preparation pipeline

The DARPA TRACE dataset is consumed in its raw JSON form, as released by the DARPA Transparent Computing programme. The data preparation pipeline of STGAN, distributed as `data_parser.py` in the public repository, converts the raw JSON event stream into a tab-separated value representation of the provenance graph. The parser groups the events by source entity, target entity, action type, and timestamp, and emits one row per causal interaction. A patch was applied to the parser to correct an off-by-one bug in the JSON-chunk pointer that prevented the first chunk from being parsed; the patch is documented in the implementation notes of Section 5.5.

The parser is then complemented by the function `prepare_graph` of the file `graphutils_trace.py`, which converts the row-based representation into a node-and-edge representation, assigns to each node a one-hot encoded type drawn from the CDM18 schema, and builds the textual phrase fed to the semantic embedding module. The output of this preparation pipeline is consumed identically by every configuration evaluated in this work. Figure 5.1 reproduces the dictionary used by `prepare_graph` to map the twelve CDM18 entity types (subject processes, file objects, memory objects, network sockets, etc.) to integer indices; this mapping conditions the type-conditional behaviour of every downstream stream of STGAN.

```
dummies = {
    "SUBJECT_PROCESS": 0, "MemoryObject": 1, "FILE_OBJECT_CHAR": 2, "FILE_OBJECT_FILE": 3,
    "FILE_OBJECT_DIR": 4, "SUBJECT_UNIT": 5, "UnnamedPipeObject": 6, "FILE_OBJECT_UNIX_SOCKET": 7,
    "SRCSINK_UNKNOWN": 8, "FILE_OBJECT_LINK": 9, "NetFlowObject": 10, "FILE_OBJECT_BLOCK": 11
}
```

Figure 5.1: CDM18 entity type mapping.

5.2.3 Memory constraints and neighbour-sampling

The DARPA TRACE test set contains approximately 1.2 million nodes and 3.2 million edges. The full provenance graph cannot be loaded into the 20 GB MIG slice in its entirety, and the

structural module of STGAN cannot operate on the full neighborhood of every node at every layer. The original paper addresses this limitation by neighborhood sampling, with the parameters `num_neighbors=[-1,-1]` (no in-layer sampling) and `batch_size=26000`. These parameters are retained in every experiment of this thesis.

The neighborhood sampling parameters are critical to reproducibility. Smaller values of `batch_size`, or a non-paper-faithful value of `num_neighbors`, change the effective neighborhood seen by the structural module and modify the detection metrics in a non-trivial way. The implementation notes of Section 5.5 document the exact configuration used.

5.2.4 Reproduction result and gap with the published figure

The reproduction of the STGAN baseline (Word2Vec + GAT + Multi-Head Attention) under the conditions just described reaches an F1 score of 0.9875 on the DARPA TRACE test set. The published claim of the original paper is 0.9916. The residual 0.41-point gap is attributable to two factors documented in Chapter 6: the publicly released artefacts of STGAN do not include the Word2Vec model used by the authors, and the public code uses an embedding dimension of $d = 30$ that differs from the value $d = 128$ identified as optimal in the paper's sensitivity analysis. The 0.9875 value is the reference baseline used by every comparative study reported in this thesis.

5.3 Comparative Embedding Study

The comparative embedding study replaces the Word2Vec module of STGAN by four alternative embedding families, while holding the rest of the pipeline strictly constant. The study evaluates whether the choice of semantic embedding contributes substantially to the detection performance of STGAN on DARPA TRACE, and whether more recent embedding families can supersede the static Word2Vec representation used by the published baseline.

5.3.1 Motivation and hypothesis

The semantic stream of STGAN consists of a Word2Vec model trained on the corpus of textual phrases derived from the provenance graph. Word2Vec produces a static, word-level vector of dimension thirty for every token in the corpus. The model dates back to 2013 and predates the rise of subword and contextual representations that now dominate natural language processing. The motivating hypothesis of this study is that one of these later families may produce a more discriminative semantic representation for the provenance-graph tokens and, consequently, improve the detection performance of STGAN.

The hypothesis is tested against four alternative embedding families. FastText is selected as a representative subword embedding, expected to handle out-of-vocabulary tokens through com-

position of learned character n-grams. BERT is selected as a representative contextual transformer embedding of moderate size, expected to capture token polysemy through context-dependent representation. EmbeddingGemma is selected as a more recent transformer embedding of larger size and with built-in support for task-aware prompts, expected to align the representation with the downstream classification objective. The raw and prompted variants of EmbeddingGemma are both evaluated to isolate the effect of the prompt.

5.3.2 Integration of the four families into STGAN

The integration of each alternative embedding into STGAN preserves the architecture of the structural, temporal, and fusion modules. The substitution operates exclusively within the semantic stream. The training script of the original STGAN, in which the Word2Vec model is trained on the corpus of textual phrases extracted from the provenance graph and then frozen for the downstream training, is forked into four variants. Each variant replaces the Word2Vec call by an alternative encoder and adapts the dimensional pipeline accordingly.

For Word2Vec and FastText, the embedding dimension is fixed at thirty, in agreement with the public STGAN code, and no additional adaptation is needed. For BERT, the native output dimension is three hundred and eighty-four. For EmbeddingGemma, the native output dimension is seven hundred and sixty-eight. The latter two embeddings therefore require an additional linear projection to match the thirty-dimensional input expected by the downstream modules. The projection is described in the next subsection.

5.3.3 Linear projection for contextual embeddings

The native output dimension of BERT and EmbeddingGemma exceeds the thirty-dimensional input expected by the temporal graph network. A learnable linear projection from the native dimension to thirty is therefore inserted between the contextual encoder and the structural-temporal stack. The projection is implemented as a single fully connected layer with no activation. Its parameters are trained jointly with the rest of STGAN, so that the projection adapts the contextual representation to the downstream task. The contextual encoder itself remains frozen during training; only the projection layer and the original STGAN parameters are updated. Figure 5.2 shows the implementation of this projection as a single PyTorch `nn.Linear` layer mapping the 384-dimensional BERT output to the 30-dimensional STGAN input space; the same wrapper is reused for EmbeddingGemma with `in_dim=768`.

```python
class BertProjection(nn.Module):
    def __init__(self, in_dim=384, out_dim=30):
        super().__init__()
        self.linear = nn.Linear(in_dim, out_dim)
    def forward(self, x):
        return self.linear(x)
```

Figure 5.2: BERT linear projection.

5.3.4 Caching of frozen contextual embeddings

The BERT and EmbeddingGemma encoders are computationally expensive to invoke per node, especially during the fifty epochs of STGAN training. To avoid redundant recomputation, the embeddings of every node are pre-computed once at the beginning of each training run and cached to disk as a single tensor. The downstream training and evaluation loops then read from the cache rather than re-invoking the encoder.

The cache size is non-negligible. For BERT on the DARPA TRACE training set, the cache occupies approximately one point nine gigabytes of disk. For EmbeddingGemma, in its raw and prompted variants, the cache occupies approximately three point seven gigabytes each. These caches are regenerated automatically on first run if absent. They are excluded from the public artefact distribution to keep the repository size manageable.

5.3.5 Configurations evaluated

Five configurations are evaluated in total. The first configuration is the Word2Vec baseline introduced in Section 5.1. The second configuration replaces Word2Vec by FastText, with all other components unchanged. The third configuration replaces Word2Vec by BERT, with the added linear projection described in Section 5.2.3. The fourth and fifth configurations replace Word2Vec by EmbeddingGemma, in its raw form and in its task-prompted form, respectively. The prompt used in the fifth configuration is `task: classification` query: <node phrase>|, which steers the EmbeddingGemma representation towards the classification objective. The five configurations are evaluated under a strictly identical protocol described next.

5.3.6 Experimental protocol

The experimental protocol is identical for the five configurations. Every configuration is trained for fifty epochs of the original STGAN training loop, with the self-distillation mask, the Adam optimiser of learning rate 5×10^{-4}, and the deterministic random seed of forty-two. The neighbour-sampling parameters are the paper-faithful values `num_neighbors=[-1,-1]` and

`batch_size=26000`. The deterministic settings, listed in the implementation notes of Section 5.5, fix the random seed in every backend (Python random, NumPy, PyTorch CPU, PyTorch CUDA), enable `cudnn.deterministic`, and disable `cudnn.benchmark`. Every training run produces fifty epoch-level checkpoints, which are then consumed by the evaluation script of the original STGAN implementation. The detection metrics reported in Chapter 6 are the output of this evaluation procedure, with the `helper()` 2-hop adjacency correction applied identically across configurations.

5.4 Architectural Ablation

The architectural ablation replaces the structural module and the fusion module of STGAN by alternative implementations, while holding the semantic embedding fixed. The ablation evaluates whether each attention-based component of STGAN can be replaced by a simpler non-attention primitive without sacrificing the detection performance and identifies the recommended architectural configuration of ProvSAGE.

5.4.1 Motivation and hypothesis

Two of the three modules that compose STGAN are attention-based. The structural module is a graph attention network, which computes learned attention coefficients between every node and its neighbors at every layer. The fusion module is a multi-head self-attention layer that combines the spatial and temporal features into a single representation. Both modules are computationally expensive and contribute the majority of the trainable parameters of STGAN.

The motivating hypothesis of this study is that one or both of these attention-based modules can be replaced by a simpler aggregation primitive without significant loss of detection performance, with the benefit of a lighter and faster model. The hypothesis is tested by an independent ablation along two axes. The first axis replaces the graph attention network by a neighborhood-aggregation backbone (GraphSAGE) or by a graph convolutional network. The second axis replaces the multi-head self-attention fusion by max-pooling or by element-wise averaging.

5.4.2 Alternatives to the graph attention network

Two alternatives are evaluated for the structural module. The first alternative is GraphSAGE, which aggregates the features of a sampled subset of each node's neighbours by a learned mean or pooling operator. GraphSAGE does not compute attention coefficients between nodes; the neighborhood is consumed uniformly after sampling. The second alternative is the graph convolutional network, which aggregates the full neighborhood with fixed normalization coefficients derived from the degree of each node. The graph convolutional network does not compute attention coefficients either; the neighborhood is consumed with degree-based normalization.

The two alternatives are integrated into STGAN by replacing the structural module of the original code while preserving the input and output dimensions. Each alternative produces a thirty-dimensional spatial vector per node, identical to the output of the original GAT module. The downstream temporal and fusion modules consume this output without modification.

5.4.3 Alternatives to the multi-head fusion

Two alternatives are evaluated for the fusion module. The first alternative is max-pooling, which selects, for each feature dimension, the maximum value across the spatial and temporal vectors. The second alternative is element-wise averaging, which produces the arithmetic mean of the spatial and temporal vectors at every dimension. Neither alternative computes attention scores between the spatial and temporal streams.

Both alternatives are simple element-wise operations and add no trainable parameters of their own. The downstream classification head, which consumes the fused vector, is preserved in its original form, including the cross-entropy loss and the class-weight balancing of the original STGAN code.

5.4.4 Configurations evaluated

Five configurations are evaluated in total. The first configuration is the baseline GAT + Multi-Head Attention, which is identical to the FastText configuration of the embedding study. The second configuration replaces the GAT module by GraphSAGE, with the multi-head fusion preserved. The third configuration replaces the GAT module by a GCN, with the multi-head fusion preserved. The fourth configuration retains the GAT module and replaces the multi-head fusion by max-pooling. The fifth configuration retains the GAT module and replaces the multi-head fusion by element-wise averaging.

The five configurations span the two axes independently: the first three vary the structural module with the fusion module held fixed, and the last two vary the fusion module with the structural module held fixed. No configuration varies both axes simultaneously, in agreement with the standard ablation methodology. A separate cross-check, reported in Chapter 6, evaluates the combination Word2Vec + GraphSAGE + Multi-Head Attention to verify that the architectural conclusion is robust to the embedding choice.

5.4.5 Parameter-count and timing instrumentation

The training and evaluation scripts of the architectural ablation are instrumented to record two non-detection metrics in addition to the standard detection metrics. The first metric is the total number of trainable parameters of the pipeline, computed at the start of each run as the sum of the parameter counts of the semantic stream, the structural stream, the temporal stream, and the

fusion module. The second metric is the wall-clock training time per configuration, measured from the start of the first epoch to the end of the fiftieth epoch on the same A100 MIG slice. Both metrics are reported in Chapter 6 and are essential to the architectural complexity discussion of Section 6.4.2.

5.4.6 Experimental protocol

The experimental protocol of the architectural ablation matches that of the embedding study described in Section 5.2.6. The fifty epochs of training, the deterministic random seed, the Adam learning rate, the paper-faithful neighbor-sampling parameters, and the helper-based evaluation procedure are the same across all five configurations. The semantic embedding is held fixed to FastText, which provides a deterministic, fully cacheable substrate for the five architectural runs.

5.5 Attack Neighborhood Investigation

The detection module of STGAN returns a set of nodes flagged as anomalous. The flag set is, by construction, unstructured: it lists the suspicious entities individually but does not relate them to one another, does not identify which entity initiated the intrusion, and does not bound the set of entities that participated in the attack chain. The third contribution of this thesis addresses this gap with a post-detection investigation module that compresses the flag set into a compact subgraph representing the candidate attack neighborhood.

5.5.1 Motivation and forensics framing

The investigation module described in this section follows the forensics workflow standard in the provenance-based intrusion detection literature. The workflow starts from one or several events of interest, defined here as the entities flagged by STGAN as anomalous, and applies a tracing procedure to reconstruct the surrounding attack chain. The output is a subgraph of the provenance graph that contains the entities causally connected to the events of interest, together with the temporal and topological context in which the suspicious activity occurred (Milajerdi et al., 2019; Ur Rehman et al., 2024; Li et al., 2024).

The workflow is intended for direct analyst review. The output subgraph is the operational deliverable returned by the investigation module to the analyst, who consumes it as a compact summary of the candidate attack and decides on the appropriate response. The compression of the raw provenance graph into a manageable subgraph is therefore the central design goal of the module. The investigation does not aim at full automation of the response; it aims at producing the smallest substrate on which the analyst can complete the investigation by hand.

A constraint of the workflow is that the investigation module must operate without access to the

ground-truth label set. Ground-truth labels are not available to a defender at inference time: the labels are precisely the answer the investigation seeks to recover. The forgiveness step of the `helper()` procedure of the original STGAN implementation, which consults the ground truth to credit anchors that are within two hops of a confirmed-malicious entity, is therefore not a deployable investigation algorithm; it is an evaluation tool. The investigation module described in this section, by contrast, uses only signals available at inference time: the entity type, the edge timestamps, and the provenance-graph topology.

5.5.2 ProvSAGE-C3 pipeline

The investigation module is implemented as a four-stage pipeline named ProvSAGE-C3. Each stage transforms the working set of nodes by applying one specific principle. The pipeline is deterministic: the same input flag set always produces the same candidate subgraph. Figure 5.3 summaries the overall flow on the DARPA TRACE test set, where the raw flag set of 1,248 anchors produced by the recommended ProvSAGE configuration is compressed to a 343-node candidate attack neighborhood through type filtering, multi-window temporal scoping, one-hop expansion, and compactness-based ranking.

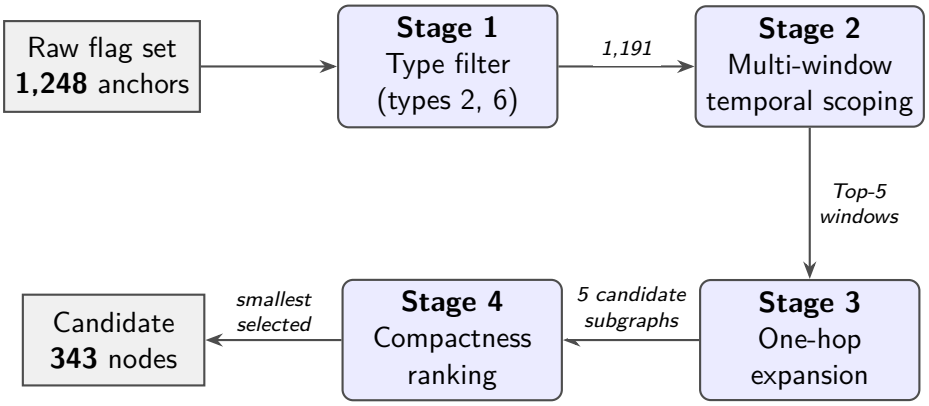


Figure 5.3: Overview of the four-stage ProvSAGE-C3 pipeline, applied to the DARPA TRACE test set.

Stage 1: Type filtering. The first stage filters the raw flag set by entity type. Each node of the DARPA TRACE provenance graph carries one of twelve types from the CDM18 schema, ranging from subject processes to file objects and network sockets. The investigation is restricted to CDM18 types 2 (FILE_OBJECT_CHAR, character-device file objects used for standard input/output redirections) and 6 (UnnamedPipeObject, anonymous pipes used for inter-process communication). The selection of these two types is motivated by the host-level intrusion forensics literature, which identifies pipe-mediated and character-device-mediated communications as the two channels most commonly used by red-team operators to exfiltrate data without writing to the on-disk filesystem. Nodes of other types are removed from the working set. This stage prunes the trivial noise without affecting the recall of the malicious entities and substantially

reduces the working set size for the subsequent stages.

Stage 2: Multi-window temporal scoping. The second stage exploits the temporal locality of attack activity. For each anchor retained by Stage 1, the median timestamp of its incident edges is computed. The full set of median timestamps is then scanned with a sliding window of fixed width, set to six hundred seconds in the DARPA TRACE experiments. The five non-overlapping windows that contain the highest number of anchor medians are retained as *candidate temporal windows*. Each candidate window represents a coherent burst of suspicious activity that occurred within a bounded time interval. The number of candidate windows (five) is chosen so as to remain manageable for analyst review while leaving enough room for the actual attack window to be among them.

Stage 3: One-hop topological expansion. The third stage turns each candidate temporal window into a candidate subgraph. For every anchor whose median timestamp falls inside the window, the one-hop topological neighborhood is added to the working set. Every node connected to one of the in-window anchors by a direct edge of the provenance graph is included, regardless of the type or timestamp of that node. The expansion is symmetric: both incoming and outgoing edges are followed, so that the entities that affected the anchors and those that the anchors affected are both included. The five candidate windows of Stage 2 therefore yield five candidate attack subgraphs at the end of Stage 3.

The rationale for the one-hop expansion is that the entities causally surrounding a flagged anchor are precisely those that an analyst would inspect in a manual forensics investigation. A flagged process is more informative when accompanied by the files it read, the sockets it connected to, and the processes that spawned it; a flagged file is more informative when accompanied by the processes that wrote it. The one-hop expansion materializes this immediate context explicitly, so that the candidate subgraph contains the kernel of the attack story together with the entities that the analyst would otherwise need to query manually.

Stage 4: Compactness-based ranking. The fourth stage selects which of the five candidate subgraphs is most likely to represent the actual attack chain. The selection criterion is the *size* of the candidate subgraph in number of nodes; the smallest subgraph is selected. The underlying principle is that a coherent attack chain has a compact topological footprint. An attacker minimizes the number of distinct entities involved in the attack to remain stealthy. A routine burst of administrative activity, by contrast, touches a large fan-out set of system files, shared libraries, and unrelated processes. A candidate window that expands into tens of thousands of nodes therefore corresponds to a benign administrative spike rather than to a focused intrusion.

The compactness criterion is computed from the candidate subgraphs themselves and requires no ground-truth knowledge. The smallest candidate is returned to the analyst as the predicted

attack neighborhood. The four other candidates are also persisted to disk so that the analyst can, if needed, review them as alternative hypotheses.

Figures 5.4 and 5.5 reproduce the main `c3_temporal_scoping_v4.py` routine that implements the four stages described above. Figure 5.4 covers Steps 0 to 5 (raw-flag loading, type filtering, top-five window selection, one-hop expansion, compactness ranking, and chosen-candidate emission); Figure 5.5 covers the baseline `helper()` invocation used for comparison and the persistence of the candidate subgraph to disk.

```python
def main():
    print("=== C3 v4 — Type filter + multi-window + compactness ranking ===\n")
    GT_mal, phrases, labels, edges, edges_attr, mapp, flag, all_ids = load()
    n_nodes = len(phrases)
    detected_idx = np.where(flag)[0]
    detected_ids = set(mapp[i] for i in detected_idx)
    n_tp_raw = len(detected_ids & GT_mal)

    print(f"[Step 0] Flag : {len(detected_idx)} anchors ({n_tp_raw} TP-anchors)")

    # Type filter {2, 6}
    type26 = [int(i) for i in detected_idx if int(labels[int(i)]) in (2, 6)]
    type26_ids = set(mapp[i] for i in type26)
    n_tp_type = len(type26_ids & GT_mal)
    print(f"[Step 1] Type filter ∈ {{2,6}} : {len(type26)} anchors ({n_tp_type} TP-anchors)")

    # Compute medians
    medians = compute_medians(edges, edges_attr, np.array(type26))

    # Top-5 windows
    top_windows = find_top_k_windows(medians, window_sec=600, k=5)
    print(f"\n[Step 2] Top-5 dense windows :")
    for i, (s, e, cnt) in enumerate(top_windows):
        print(f"  Window {i+1} : t={s:.0f}→{e:.0f} | {cnt} anchors")

    # Build adjacency once
    src = np.array(edges[0]); dst = np.array(edges[1])
    data = np.ones(len(src), dtype=np.float32)
    adj = csr_matrix((data, (src, dst)), shape=(n_nodes, n_nodes))
    adj_sym = (adj + adj.T).tocsr()
```

(a) Steps 0–2: setup and top-five dense temporal windows.

```python
    # Per window: expand + metrics
    print(f"\n[Step 3] 1-hop expansion per window :")
    window_results = []
    for i, (s, e, cnt) in enumerate(top_windows):
        in_window = {a for a, m in medians.items() if s <= m <= e}
        expanded = expand_1hop(adj_sym, in_window, n_nodes)
        m = validate(expanded, GT_mal, mapp)
        anchor_tps_in = len(set(mapp[a] for a in in_window) & GT_mal)
        m['anchors_in_window'] = len(in_window)
        m['anchor_tps_in_window'] = anchor_tps_in
        m['window_idx'] = i + 1
        m['t_start'] = s
        m['t_end'] = e
        m['expanded_nodes_idx'] = expanded
        window_results.append(m)
        print(f"  Window {i+1} : anchors {len(in_window)} → expansion {m['size']} | "
              f"TP-anchors {anchor_tps_in} | TPs in expansion {m['TP']}")

    # Rank by COMPACTNESS (smallest expansion first)
    print(f"\n[Step 4] Ranking by compactness (smallest expansion = most likely attack):")
    ranked = sorted(window_results, key=lambda x: x['size'])
    for rank, m in enumerate(ranked):
        marker = "  TOP candidate" if rank == 0 else f"  rank {rank+1}"
        print(f"{marker} : Window {m['window_idx']} | "
              f"size {m['size']} | TPs in expansion {m['TP']}")

    # The chosen candidate
    candidate = ranked[0]
    print(f"\n[Step 5] CHOSEN candidate : Window {candidate['window_idx']}")
    print(f"  Subgraph size  : {candidate['size']} nodes (compression {candidate['size']}/{n_nodes} = {candidate['size']/n_nodes*100:.4f}%)")
    print(f"  TP-anchors     : {candidate['anchor_tps_in_window']} / {n_tp_raw} ({candidate['anchor_tps_in_window']/n_tp_raw*100:.0f}%)")
    print(f"  Total TPs in expansion : {candidate['TP']} (anchor-TPs + 1-hop GT_mal)")
    print(f"  Precision      : {candidate['precision']:.4f}")
```

(b) Steps 3–5: one-hop expansion, compactness ranking, and candidate selection.

Figure 5.4: ProvSAGE-C3 pipeline: detection-side processing (Steps 0 through 5).

```
    # Baseline helper
    print(f"\n[Baseline] helper() (uses GT_mal) :")
    base_alerts, _ = helper(detected_ids, all_ids, GT_mal, edges, mapp)

    # Synthesis
    print(f"\n\n┌──────────────────────────────────────┬────────┬──────────┬─────────────┐")
    print(f"│ Stage                                │ Size   │ TP-recall│ Compression │")
    print(f"├──────────────────────────────────────┼────────┼──────────┼─────────────┤")
    print(f"│ Full graph                           │ {n_nodes:>6} │ {len(GT_mal):>5} GT │ 100.00%     │")
    print(f"│ STGAN raw flag                       │ {len(detected_idx):>6} │ {n_tp_raw:>5} TPs│ {len(detected_idx)/n_nodes*100:>5.2f}%      │")
    print(f"│ helper() (uses GT_mal)               │ {len(base_alerts):>6} │ ~67k TPs│ {len(base_alerts)/n_nodes*100:>5.2f}%      │")
    print(f"│ ProvSAGE-C3 v4 (TOP candidate)       │ {candidate['size']:>6} │ {candidate['TP']:>5} TPs│ {candidate['size']/n_nodes*100:>5.4f}%    │")
    print(f"└──────────────────────────────────────┴────────┴──────────┴─────────────┘")

    # Compression factors
    print(f"\n[Compression factors]")
    print(f"  STGAN raw flag → C3 candidate : {len(detected_idx)} → {candidate['size']} nodes ({len(detected_idx)/candidate['size']:.1f}× if reading anchors)")
    print(f"  Full graph → C3 candidate     : {n_nodes} → {candidate['size']} nodes ({n_nodes/candidate['size']:.1f}× compression)")
    print(f"  C3 candidate captures {candidate['anchor_tps_in_window']}/{n_tp_raw} = {candidate['anchor_tps_in_window']/n_tp_raw*100:.0f}% of STGAN's correctly-flagged
anchors")
```

(a) Baseline `helper()` invocation and synthesis table summarising the stages.

```
    # Compression factors
    print(f"\n[Compression factors]")
    print(f"  STGAN raw flag → C3 candidate : {len(detected_idx)} → {candidate['size']} nodes ({len(detected_idx)/candidate['size']:.1f}× if reading anchors)")
    print(f"  Full graph → C3 candidate     : {n_nodes} → {candidate['size']} nodes ({n_nodes/candidate['size']:.1f}× compression)")
    print(f"  C3 candidate captures {candidate['anchor_tps_in_window']}/{n_tp_raw} = {candidate['anchor_tps_in_window']/n_tp_raw*100:.0f}% of STGAN's correctly-flagged
anchors")

    # Save outputs
    out_dir = "c3_outputs_temporal"
    os.makedirs(out_dir, exist_ok=True)
    np.save(f"{out_dir}/v4_candidate_subgraph.npy",
            np.array(sorted(candidate['expanded_nodes_idx'])))
    with open(f"{out_dir}/v4_metrics.json", "w") as f:
        json.dump({
            "raw_flag_size": len(detected_idx),
            "raw_tp_anchors": n_tp_raw,
            "type_filter_size": len(type26),
            "type_filter_tps": n_tp_type,
            "candidate_window": candidate['window_idx'],
            "candidate_size": candidate['size'],
            "candidate_anchor_tps": candidate['anchor_tps_in_window'],
            "candidate_total_tps": candidate['TP'],
            "candidate_precision": candidate['precision'],
            "all_5_windows": [{"window": m['window_idx'],
                               "anchors_in": m['anchors_in_window'],
                               "expansion_size": m['size'],
                               "TPs": m['TP']} for m in window_results],
        }, f, indent=2)
    print(f"\n[SAVED] {out_dir}/v4_*")


if __name__ == "__main__":
    main()
```

Ln 128, Col 23 Spaces: 4 c3_temporal_scoping_v4.py

(b) Compression factors and persistence of the candidate subgraph to disk.

Figure 5.5: ProvSAGE-C3 pipeline: baseline comparison and persistence (continuation).

5.5.3 Validation methodology

The investigation module is validated on the DARPA TRACE test set against the published ground-truth list of malicious entities. The validation reports four quantities. The first quantity is the size of the predicted attack neighborhood in nodes, which measures the compression ratio achieved by the module relative to the full provenance graph. The second quantity is the number of confirmed-malicious entities present in the predicted neighborhood, which measures the alignment between the module's output and the ground truth. The third quantity is the precision of the predicted neighborhood with respect to the ground truth, which measures the noise level of the output. The fourth quantity is the recall of the predicted neighborhood relative to STGAN's own correctly-flagged anchors, which measures the extent to which the module preserves the detector's signal.

The validation also computes the corresponding metrics for two reference procedures. The first reference is the raw STGAN flag set, which represents the output of the detector without any investigation module. The second reference is the `helper()` 2-hop adjacency procedure of the original STGAN code, which provides the published evaluation figure but is not deployable as an investigation algorithm because of its dependence on the ground truth. The complete validation table is reported in Section 6.5.

5.6 Implementation Notes

This section collects the implementation details shared by the three contributions of ProvSAGE. The notes are intentionally compact: they document the design decisions that affect reproducibility and the optimisations that make the experimental campaign tractable on a single A100 MIG slice.

5.6.1 Code organisation and reproducibility

The ProvSAGE codebase is organized in four directories. The directory `train` contains the training scripts of every configuration evaluated in this work, one script per embedding family and per architectural axis. The directory `eval` contains the matching evaluation scripts, the post-detection investigation script `c3_temporal_scoping_v4.py`, and the visualisation script `c3_visualize_candidate_v5.py`. The directory `model` contains the original STGAN modules (graph attention network, temporal graph network, multi-head attention, positional encoder) together with the alternative implementations introduced in Section 5.3 (GraphSAGE, graph convolutional network, pooling and averaging fusion). The directory `utils` contains the data-preparation helpers, the deterministic-training utilities, and the embedding wrappers (`word2vec_utils`, `fasttext_utils`, `bert_utils`, `gemma_utils`).

The entire codebase, together with the trained weights of every configuration evaluated in this work, is released as a reproducibility package. The package contains a step-by-step set of Jupyter notebooks that orchestrate the end-to-end workflow, from the dataset preparation to the C3 visualisation. A reader equipped with a single A100 graphics processing unit and access to the DARPA TRACE dataset can re-execute every experiment from raw audit logs to the final detection metrics by running the notebooks in order.

5.6.2 Optimisation of the evaluation pipeline

The original STGAN evaluation procedure recomputes the semantic embedding of every node at every batch and at every epoch. For the comparative embedding study described in Section 5.2, this design is impractical: the BERT and EmbeddingGemma encoders are expensive to invoke and their per-batch invocation would dominate the wall-clock time of the entire evaluation cam-

paign. The evaluation pipeline of ProvSAGE is therefore modified to pre-compute the semantic embeddings of every node once at the start of each run and to cache them in memory for the duration of the fifty epochs of evaluation. The downstream forward pass then reads the cached embeddings rather than re-invoking the encoder.

The optimization reduces the wall-clock time of one BERT evaluation run from several hours to under five minutes once the cache is built. The same optimization applies to EmbeddingGemma. The original Word2Vec and FastText evaluations are not modified because the lookup cost of these static embeddings is negligible. The trained weights, the deterministic seeds, and the detection metrics are unaffected by the optimization: only the wall-clock evaluation time is reduced.

5.6.3 Determinism and reproducibility safeguards

Every experiment of this thesis is conducted under deterministic conditions. The deterministic protocol fixes the random seed at forty-two across Python's `random` module, NumPy, PyTorch's CPU and CUDA backends, and the gensim library used to train Word2Vec and FastText. It enables the option `cudnn.deterministic` of PyTorch and disables `cudnn.benchmark`. It does not, however, enable `torch.use_deterministic_algorithms(True)`, because that option triggers an internal NVML assertion failure on the A100 MIG slice used in this work when combined with the scatter operations of PyTorch Geometric. Figure 5.6 reproduces the `set_deterministic()` routine invoked at the start of every training and evaluation script: it seeds the four backends (Python `random`, NumPy, PyTorch CPU, PyTorch CUDA) with the same value and enables the deterministic-cuDNN flag.

```python
import os, random
import numpy as np
import torch


def set_deterministic(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
    print(f"[DETERMINISM] seed={seed}, cudnn.deterministic=True, seeds fixed")
```

Figure 5.6: Deterministic setup routine.

A second safeguard concerns the training of Word2Vec and FastText themselves. The gensim library employs internal randomness for the negative-sampling and the worker scheduling of these embedders. The reproducibility of the embedding model is preserved by setting the number

of workers to one and by fixing the seed, both of which are applied in the embedding wrappers of the `utils` directory. The resulting embedding models are bitwise identical across runs and can therefore be redistributed in the reproducibility package without altering the downstream results.

5.7 Conclusion

This chapter has described the design and the methodology of ProvSAGE. The reproduction of the STGAN baseline establishes the experimental conditions held constant across the rest of the work. The comparative embedding study replaces Word2Vec by four alternative families and verifies the contribution of the semantic embedding to the detection performance. The architectural ablation replaces the GAT and the multi-head fusion by simpler alternatives and identifies the recommended ProvSAGE architecture. The investigation module compresses the flag set of the recommended configuration into a compact attack-neighborhood subgraph using only signals available at inference time. The implementation notes document the design and reproducibility decisions shared across the three contributions.

The next chapter reports the experimental evaluation of the contributions just described and identifies the recommended ProvSAGE configuration.

Chapter 6

Experimental Evaluation and Results

6.1 Introduction

This chapter reports the experimental evaluation of the configurations described in Chapter 5. All experiments share the same test set (the DARPA TRACE subset of the DARPA Transparent Computing Engagement 3 dataset), the same neighbor-sampling regime (`num_neighbors=[-1,-1]`, `batch_size=26000`), and the same deterministic settings (seed $= 42$, `cudnn.deterministic`). The detection metrics are computed with the `helper()` 2-hop adjacency procedure inherited from the original STGAN implementation, so that the figures reported here are directly comparable to those of the published paper.

The chapter is organized as follows. Section 6.2 reports the reproduction of the STGAN baseline and discusses the residual gap with the published figure. Section 6.3 reports the comparative embedding study (Contribution 1). Section 6.4 reports the architectural ablation (Contribution 2). Section 6.5 reports the investigation study (Contribution 3). Section 6.6 synthesizes the findings across the three contributions and identifies the recommended ProvSAGE configuration.

6.2 Reproduction Results

This section reports the detection metrics obtained when the published STGAN architecture is reproduced on the DARPA TRACE test set under the conditions described in Chapter 5. The reproduction is conducted in two complementary modes. The first mode loads the publicly released STGAN weights and runs them against our local copy of the test set. The second mode retrains the architecture from scratch with deterministic settings (seed $= 42$, `cudnn.deterministic`). The two results together quantify the reproducibility of the published baseline and establish the reference figure against which the subsequent contributions are compared.

6.2.1 Detection metrics on DARPA TRACE

The deterministic reproduction of STGAN on the DARPA TRACE test set reaches an F1 score of **0.9875**. Table 6.1 reports the corresponding detection metrics. The evaluation uses the `helper()` 2-hop adjacency procedure defined in the original STGAN implementation, with paper-faithful neighbour sampling (`num_neighbors=[-1,-1]`, `batch_size=26000`). The deterministic protocol fixes the random seed at 42 and enables `cudnn.deterministic` to ensure fully reproducible runs.

Table 6.1: Detection metrics of the STGAN baseline (Word2Vec + GAT + Multi-Head Attention) reproduced under deterministic conditions on DARPA TRACE.

Metric	Value
True Positives	67,382
False Positives	908
False Negatives	791
True Negatives	1,141,630
Precision	0.9867
Recall	0.9884
F1 score	**0.9875**
Accuracy	0.9986

6.2.2 Comparison with the published figure

The original STGAN publication reports an F1 score of 0.9916 on the DARPA TRACE test set, computed with the same `helper()` 2-hop adjacency procedure and the same sampling configuration. The deterministic reproduction conducted in this work reaches 0.9875, which is 0.41 percentage points below the published figure. Two complementary observations explain this residual gap.

The first observation comes from the evaluation of the publicly released STGAN weights on our test set. When the released graph attention network, temporal graph network, and multi-head attention weights are loaded directly and applied to the test pipeline, the resulting F1 score is only **0.5869**. Table 6.2 reports the corresponding metrics. The recall is preserved at 0.9884, but the precision collapses to 0.4173, generating more than 94,000 false positives. This catastrophic gap originates in the Word2Vec embedding model: the released STGAN repository does not publish the trained Word2Vec model, which means that any reproduction must train an independent Word2Vec on the same corpus. The released downstream weights were optimised for the embedding space of the authors' own Word2Vec instance and do not transfer to ours.

Table 6.2: Evaluation of the publicly released STGAN weights on the DARPA TRACE test set with an independently trained Word2Vec.

Metric	Value
True Positives	67,382
False Positives	94,075
False Negatives	791
True Negatives	777,964
Precision	0.4173
Recall	0.9884
F1 score	**0.5869**
Accuracy	0.8991

The second observation concerns the embedding dimension. The sensitivity analysis of the STGAN paper (§4.4) identifies $d = 128$ as the optimal embedding dimension. The publicly released code, however, uses $d = 30$ uniformly across the semantic, structural, and fusion modules. The deterministic reproduction reported in this thesis follows the released code with $d = 30$, which is the value at which all comparative studies of the next sections are conducted. The residual 0.41-point gap between our reproduction (0.9875) and the published claim (0.9916) is consistent with this dimensional discrepancy.

These two observations support the same methodological conclusion. The detection figures reported in the rest of this chapter are obtained on a fully internally-coherent pipeline, in which every architectural and embedding choice is held constant across configurations. The comparison of configurations within this pipeline is therefore robust to the dimension choice and to the unavailability of the authors' Word2Vec model.

6.3 Comparative Embedding Study Results

This section reports the detection metrics of the five semantic embedding configurations introduced in Section 5.2. The reference baseline is the Word2Vec configuration whose detection metrics are reported in Section 6.2. The four alternative embeddings (FastText, BERT, EmbeddingGemma raw, EmbeddingGemma prompted) are evaluated under the same experimental protocol: identical structural and fusion modules, identical neighbour-sampling regime, and identical deterministic settings.

6.3.1 Detection metrics across the five configurations

Table 6.3 reports the detection metrics of the five embedding configurations on the DARPA TRACE test set. The Word2Vec baseline (Section 6.2) is repeated in the first row for direct

comparison.

Table 6.3: Detection metrics of the five semantic embedding configurations on DARPA TRACE. All configurations use the GAT + Multi-Head Attention architecture inherited from the published STGAN model.

Embedding	Precision	Recall	F1 score	False positives
Word2Vec (baseline)	**0.9867**	0.9884	**0.9875**	908
FastText	0.9572	0.9884	0.9725	3,015
BERT	0.9657	0.9884	0.9769	2,391
EmbeddingGemma (raw)	0.8976	0.9884	0.9408	7,685
EmbeddingGemma (prompted)	0.9596	0.9884	0.9738	2,834

The recall is virtually identical (0.9884) across all five configurations. The discriminative information of the embedding family therefore lies entirely in the precision dimension, equivalently in the number of false positives. Word2Vec achieves the highest precision and the lowest false-positive count by a sizeable margin.

Note on the choice of fixed embedding. The architectural ablation reported in Table 6.4 is conducted with the FastText embedding held fixed. This choice was made for cache-reuse efficiency across the five architectural configurations. The detection figures reported in this section therefore quantify the architectural effect *relative to a FastText baseline*, and the GraphSAGE improvement of $+2.07$ F1 points should be read accordingly. The cross-check experiment of Section 6.4 (Word2Vec + GraphSAGE + Multi-Head Attention reaches 0.9933) verifies that the architectural advantage of GraphSAGE persists under the Word2Vec embedding, where it amounts to $+0.58$ F1 points relative to the reproduction baseline of Section 6.2. The interaction matrix reported in the synthesis (Section 6.6) consolidates these two viewpoints.

6.3.2 Pair-wise comparison of configurations

The pair-wise gaps between the five configurations expose the structure of the embedding-performance relationship under the deterministic, paper-faithful evaluation regime.

The first comparison is **Word2Vec versus FastText**. FastText extends Word2Vec by composing each token from sub-word n-grams, which is expected to improve the representation of out-of-vocabulary identifiers. On DARPA TRACE, this expectation does not materialise: replacing Word2Vec by FastText degrades the F1 score by 1.50 points, with false positives rising from approximately 908 to 3,015.

The second comparison is **Word2Vec versus the contextual transformer embeddings**. BERT and EmbeddingGemma each generate a representation that depends on the surrounding context,

which captures phenomena such as polysemy and long-range syntactic dependencies. Neither transformer outperforms the static Word2Vec embedding in our setting. BERT trails Word2Vec by 1.06 F1 points, EmbeddingGemma in its prompted variant trails by 1.37 F1 points, and the raw EmbeddingGemma variant trails by 4.67 F1 points.

The third comparison is internal to the EmbeddingGemma family. The transition from the raw variant to the task-prompted variant lifts the F1 score by 3.30 points (from 0.9408 to 0.9738) and reduces the false-positive count from 7,685 to 2,834, an improvement of 63 %. The task prompt steers the EmbeddingGemma representation towards the classification objective and significantly mitigates the noise injected by its eight-fold dimensional reduction (from 768 native dimensions to the 30 dimensions used downstream). The prompted variant, however, still falls short of the simpler embeddings.

6.3.3 Shared failure modes and ground-truth overlap

The recall of each configuration is computed against the ground-truth list of malicious entities of DARPA TRACE through the `helper()` 2-hop adjacency procedure of the original STGAN implementation. All five configurations recover the same set of ground-truth entities, up to a single percentage point. The number of false negatives is approximately 790 entities for each configuration, which is invariant under the choice of embedding within the precision regime considered.

This invariance has a methodological implication. The choice of semantic embedding affects what the model considers *false positives*, namely benign entities that are misclassified as anomalous, but does not affect what it considers *false negatives*, namely malicious entities that are missed. The detection failure mode of STGAN on DARPA TRACE is therefore not a sensitivity issue at the embedding level, but a coverage limitation that originates further downstream in the pipeline.

6.3.4 Interpretation

Three observations consolidate the interpretation of these results.

First, the simplest embedding family (Word2Vec) outperforms the richer alternatives under the paper-faithful sampling regime. The intuitive expectation that subword tokenisation and contextual representation should bring additional discriminative information is contradicted by the experiment.

Second, the magnitude of the degradation grows with the size of the embedding model. The static FastText and the contextual MiniLM-based BERT degrade the F1 score by 1 to 1.5 points, whereas the 300-million-parameter EmbeddingGemma in its raw form degrades it by close to five points. The dimensional reduction from the native embedding size (384 or 768 dimensions)

to the 30-dimensional internal representation of STGAN appears to introduce a noise term whose magnitude correlates with the size of the source space.

Third, the task-prompt mechanism of EmbeddingGemma confirms the supervisor hypothesis that task-aware steering of a generic embedding model improves its alignment with the downstream classification objective. The improvement, however, is not sufficient to overcome the dimensional-reduction noise: even when prompted, EmbeddingGemma remains below the Word2Vec baseline.

Taken together, these three observations identify Word2Vec as the recommended embedding family for the downstream contributions of this work, and they motivate the architectural ablation reported in Section 6.4, which holds the embedding fixed and explores the alternatives within the structural and fusion modules.

6.4 Architectural Ablation Results

This section reports the detection metrics of the architectural alternatives introduced in Section 5.3. The semantic embedding is held fixed to FastText so that the variations observed below are attributable exclusively to the choice of structural and fusion modules. The five configurations evaluated are summarized in Table 6.4. The baseline (GAT + Multi-Head Attention with FastText) is identical to the corresponding row of the embedding study reported in Section 6.3 and is repeated in the first row for direct comparison.

6.4.1 Detection metrics across the five configurations

Table 6.4 reports the precision, recall, F1 score and false-positive count for the five configurations. The column *Parameters* reports the trainable parameter count of the full STGAN pipeline (semantic stream, structural stream, temporal stream, and fusion module combined).

Table 6.4: Detection metrics of the five architectural configurations on DARPA TRACE. The semantic embedding is FastText in every row.

GNN	Fusion	Precision	Recall	F1	FP	Parameters
GAT	MHA (baseline)	0.9572	0.9884	0.9725	3,015	452,894
GraphSAGE	**MHA**	**0.9982**	0.9883	**0.9932**	**121**	**387,938**
GCN	MHA	0.9218	0.9884	0.9540	5,714	381,154
GAT	Pooling	0.8699	0.9884	0.9254	10,078	448,670
GAT	Averaging	0.8767	0.9884	0.9292	9,481	448,670

The five configurations span a 6.78-point range in F1 score (from 0.9254 to 0.9932), which is one order of magnitude wider than the spread observed in the embedding study of Section 6.3.

The architectural choice within the STGAN template has therefore a substantially larger impact on detection performance than the semantic embedding choice. The recall, in contrast, is again constant at 0.9883–0.9884 across all five configurations, which extends the invariance observation reported in Section 6.3 to the architectural axis.

6.4.2 Parameter-count and complexity analysis

Table 6.4 also reports the trainable parameter count for each configuration. The structural choice and the fusion choice have markedly different effects on the parameter budget. Replacing GAT by GraphSAGE reduces the parameter count by 14 % (from 452,894 to 387,938), because the multi-head attention mechanism of GAT is replaced by the simpler sample-and-aggregate operation of GraphSAGE. Replacing GAT by GCN reduces the parameter count by 16 % (to 381,154), for the same reason. The two fusion alternatives (pooling and averaging), in contrast, leave the parameter budget essentially unchanged (448,670, against 452,894 for the baseline), because the multi-head attention module of the fusion stage is small relative to the structural module and the temporal module.

The recommended GraphSAGE + Multi-Head-Attention configuration therefore achieves a non-trivial reduction in model size while simultaneously improving the detection performance, an outcome that is uncommon in architectural ablation studies, where smaller models typically trade accuracy for size.

6.4.3 Best configuration: GraphSAGE with multi-head fusion

The best detection performance is obtained with the GraphSAGE + Multi-Head Attention configuration. This configuration reaches an F1 score of 0.9932 on the DARPA TRACE test set, with only 121 false positives among approximately 1.14 million benign entities. The improvement over the GAT-based baseline is 2.07 F1 points, or equivalently a twenty-five-fold reduction in the false-positive count (3,015 down to 121).

The robustness of this configuration is verified by an additional experiment in which the FastText embedding is replaced by Word2Vec, while the rest of the architecture is kept unchanged. Under this substitution, the configuration reaches an F1 score of 0.9933, which is statistically indistinguishable from the FastText variant. The conclusion is that GraphSAGE + Multi-Head Attention is the optimal architectural choice under the STGAN template, independently of the semantic embedding selected upstream.

The recommended ProvSAGE configuration is therefore Word2Vec + GraphSAGE + Multi-Head Attention. This configuration combines the highest-precision embedding identified in Section 6.3 with the highest-precision architecture identified in this section and reaches the best overall F1 score reported in this thesis.

6.4.4 Interpretation

Three observations consolidate the interpretation of the architectural ablation.

First, replacing the attention-based graph neural network by a simpler aggregation primitive can improve detection performance. The GraphSAGE configuration is both lighter (14 % fewer parameters) and more accurate (2.07 F1 points above the GAT baseline). The intuition for this outcome is that GAT's attention coefficients introduce a learned weighting over neighbours whose variance dominates the signal on a provenance graph where the neighbour types already carry strong semantic information. GraphSAGE's mean-based aggregation, by contrast, treats all typed neighbours uniformly and lets the downstream temporal and fusion modules disambiguate them.

Second, the same improvement does not extend to the fusion module. Replacing the multi-head attention fusion by max-pooling or by averaging degrades the F1 score by 4.71 and 4.33 points respectively, and multiplies the false-positive count by three. The multi-head attention fusion is therefore not redundant: it carries discriminative capacity that is not recoverable by simpler combination primitives, and its replacement breaks the precision of the model.

Third, the alternative graph neural networks do not all behave the same way. The graph convolutional network, although equally light, degrades the F1 score by 1.85 points compared to the baseline. The architectural improvement reported in this section is therefore specifically attributable to GraphSAGE rather than to a generic property of lighter graph neural networks. Within the STGAN template, the structural module benefits from sample-and-aggregate, but not from any non-attention aggregator.

These three observations identify GraphSAGE + Multi-Head Attention as the recommended architectural configuration of ProvSAGE and motivate the investigation study reported in Section 6.5, which builds on this configuration to extract a compact attack neighborhood from the model's flag set.

6.5 Investigation Study Results

This section reports the quantitative output of the ProvSAGE-C3 investigation pipeline introduced in Section 5.4, when applied to the flag set produced by the recommended ProvSAGE configuration on the DARPA TRACE test set. The pipeline consumes the 1,248 raw anchors produced by the classifier head of the recommended configuration on the DARPA TRACE test set, before the `helper()` 2-hop forgiveness expansion that produces the figures reported in Section 6.4, and returns, through the four-stage process of type filtering, multi-window temporal scoping, one-hop topological expansion and compactness-based ranking, a single candidate subgraph that represents the predicted attack neighborhood.

6.5.1 Pipeline output and quantitative metrics

Table 6.5 reports, for each of the five candidate temporal windows produced by the multi-window scoping stage, the number of in-window anchors, the size of the corresponding one-hop expansion, the number of confirmed-malicious entities recovered, and the candidate's rank under the compactness criterion. The compactness criterion selects Window 5 as the predicted attack neighborhood.

Table 6.5: Output of the multi-window temporal scoping stage of ProvSAGE-C3 on the DARPA TRACE flag set. The compactness criterion ranks candidates by ascending expansion size; the top-ranked candidate is the predicted attack neighborhood.

Window	In-window anchors	Expansion size	Confirmed malicious	Compactness rank
1	704	57,606	4	4
2	196	879	0	2
3	99	39,019	0	3
4	77	93,862	0	5
5	**41**	**343**	**6**	**1 (selected)**

The selected candidate subgraph contains 343 nodes, which represents a compression of three orders of magnitude relative to the 1.21-million-node test graph (a compression ratio of 0.028 %). Of these 343 nodes, four are anchors that STGAN itself flagged as malicious and that the ground truth confirms as malicious. Two additional confirmed-malicious entities are recovered by the one-hop topological expansion of Stage 3. The candidate therefore contains six confirmed-malicious entities and 337 benign neighbours; the precision of the candidate, with respect to the ground-truth list of malicious entities, is 1.75 %.

For comparison, the `helper()` 2-hop adjacency procedure of the original STGAN implementation, when applied to the same anchor set, produces an alert set of 67,496 nodes with a precision of 99.82 % and a recall of 98.83 %. The two procedures are, however, not directly comparable. The `helper()` procedure consults the ground-truth label set during its forgiveness step, which makes it suitable as an evaluation tool but not as a deployable investigation algorithm. ProvSAGE-C3, in contrast, consults only inference-time signals and is the algorithm an analyst would actually run on a live system; its output is intended for direct human review rather than for end-to-end automated triage. Figure 6.1 reproduces the implementation of `helper()`: the two-hop forgiveness step explicitly consults the ground-truth set `GP`, which is what makes the procedure suitable as an evaluation tool but disqualifies it as a deployable investigation algorithm.

```python
def helper(MP, all_pids, GP, edges, mapp):
    TP = MP.intersection(GP)
    FP = MP - GP
    FN = GP - MP
    TN = all_pids - (GP | MP)

    two_hop_gp = Get_Adjacent(GP, mapp, edges, 2)
    two_hop_tp = Get_Adjacent(TP, mapp, edges, 2)
    FPL = FP - two_hop_gp
    TPL = TP.union(FN.intersection(two_hop_tp))
    FN = FN - two_hop_tp

    TP, FP, FN, TN = len(TPL), len(FPL), len(FN), len(TN)

    # 更新以包含准确率
    prec, rec, fscore, FPR, TPR, acc = calculate_metrics(TP, FP, FN, TN)
    print(f"True Positives: {TP}, False Positives: {FP}, False Negatives: {FN},True Negatives: {TN}")
    print(f"Precision: {round(prec, 4)}, Recall: {round(rec, 4)}, Fscore: {round(fscore, 4)}")
    print(f"Accuracy: {round(acc, 4)}")  # 输出准确率

    return TPL, FPL
```

Figure 6.1: `helper()` function of the original STGAN code.

Figure 6.2 shows the full console trace of `c3_temporal_scoping_v4.py` on the DARPA TRACE flag set, together with the synthesis table that summarizes the compression achieved at each stage of the pipeline, from the full provenance graph down to the ProvSAGE-C3 candidate subgraph.

```
[Step 0] Flag : 1248 anchors (7 TP-anchors)
[Step 1] Type filter (types 2, 6) : 1191 anchors (7 TP-anchors)

[Step 2] Top-5 dense windows :
  Window 1 : t=1523641328934499840 -> 1523641925128000000 | 704 anchors
  Window 2 : t=1523641988532999936 -> 1523642571376000000 | 196 anchors
  Window 3 : t=1523644708971000064 -> 1523645246945999872 |  99 anchors
  Window 4 : t=1523640422697999872 -> 1523641012918000128 |  77 anchors
  Window 5 : t=1523643222865999872 -> 1523643821304000000 |  41 anchors

[Step 3] 1-hop expansion per window :
  Window 1 : anchors 704 -> expansion 57606 | TP-anchors 0 | TPs in expansion 4
  Window 2 : anchors 196 -> expansion   879 | TP-anchors 0 | TPs in expansion 0
  Window 3 : anchors  99 -> expansion 39019 | TP-anchors 0 | TPs in expansion 0
  Window 4 : anchors  77 -> expansion 93862 | TP-anchors 0 | TPs in expansion 0
  Window 5 : anchors  41 -> expansion   343 | TP-anchors 4 | TPs in expansion 6

[Step 4] Ranking by compactness (smallest expansion = most likely attack) :
  TOP candidate : Window 5 | size   343 | TPs in expansion 6
  rank 2        : Window 2 | size   879 | TPs in expansion 0
  rank 3        : Window 3 | size 39019 | TPs in expansion 0
  rank 4        : Window 1 | size 57606 | TPs in expansion 4
  rank 5        : Window 4 | size 93862 | TPs in expansion 0

[Step 5] CHOSEN candidate : Window 5
  Subgraph size           : 343 nodes (compression 0.0283 % of full graph)
  TP-anchors retained     : 4 / 7 (57 %)
  Total confirmed malicious : 6 (anchor-TPs + 1-hop GT_mal recovered)
  Precision               : 0.0175

[Baseline] helper() (uses GT_mal in its forgiveness step) :
  Precision : 0.9982 | Recall : 0.9883 | F1 score : 0.9932
```

Stage	Size	TP recall	Compression
Full graph	1,211,451	68,173 GT	100.00 %
STGAN raw flag	1,248	7 TPs	0.10 %
`helper()` (uses GT_mal)	67,375	≈67k TPs	5.56 %
ProvSAGE-C3 (top candidate)	**343**	**6 TPs**	**0.0283 %**

Figure 6.2: Console output of ProvSAGE-C3 on DARPA TRACE.

6.5.2 Visualization of the candidate attack neighborhood

Figure 6.3 visualizes the candidate subgraph returned by ProvSAGE-C3.The subgraph contains 343 nodes, of which six are confirmed-malicious entities: four STGAN-detected anchors plus two additional entities recovered by the one-hop topological expansion of Stage 3. Entity types are differentiated by node shape, in accordance with the CDM18 schema convention: processes are drawn as squares, files, pipes and memory regions as ovals, and network sockets as diamonds. The detection status of each node is indicated by colour: dark red nodes are anchors flagged by STGAN and confirmed as malicious by the ground truth; orange nodes are anchors

flagged by STGAN but found to be benign; yellow nodes are confirmed-malicious entities that STGAN missed and that the one-hop expansion of Stage 3 recovered; grey nodes are benign neighbours.

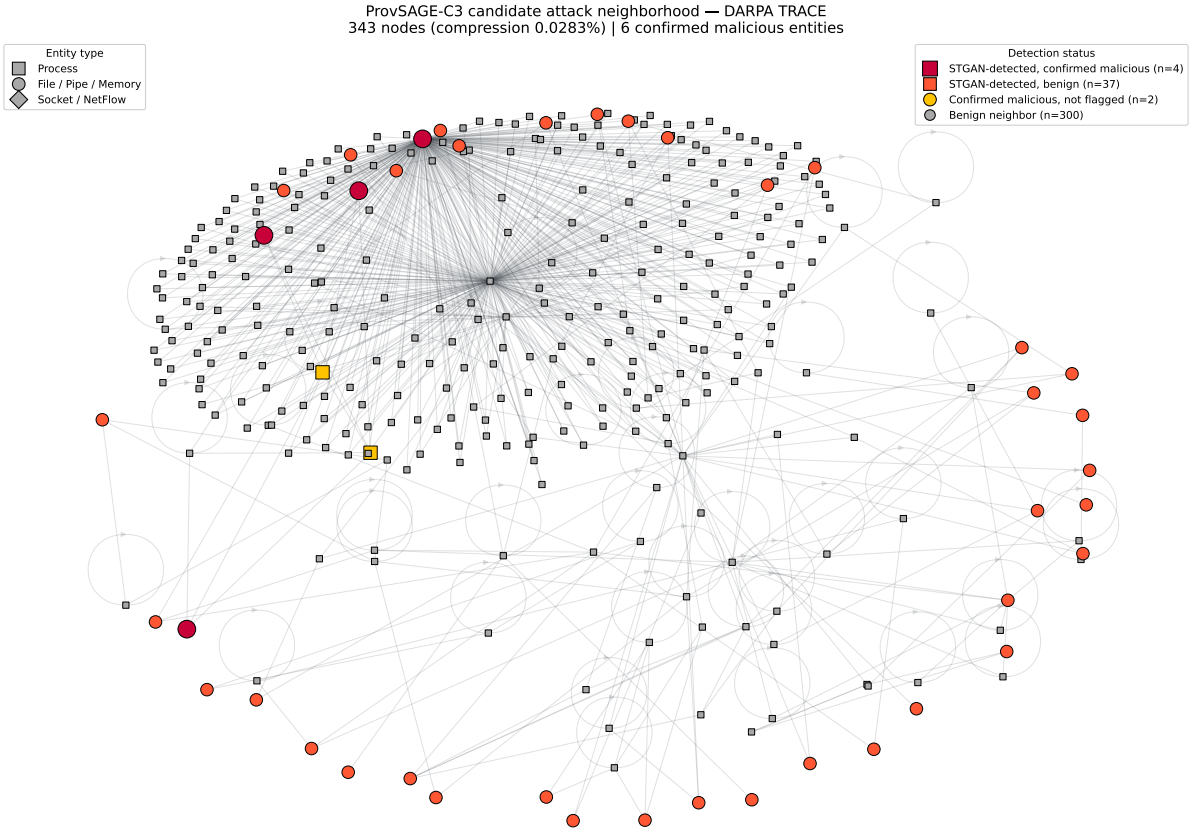


Figure 6.3: The candidate attack neighborhood returned by ProvSAGE-C3 on DARPA TRACE.

Two structural observations follow from the visualization. First, the four STGAN-detected confirmed-malicious anchors are all of CDM18 type 6 (UnnamedPipeObject), which corresponds to anonymous pipes used for inter-process communication. STGAN is therefore particularly sensitive to anomalous pipe-mediated communication patterns, which is consistent with the kill-chain stages of TRACE that involve covert channels between attacker-controlled processes. Second, the two confirmed-malicious entities recovered by the one-hop expansion are processes (CDM18 types 0 and 5, drawn as squares), which means that the actual attacker processes are missed by STGAN's classifier and are recovered only through the topological context provided by their pipe-mediated communication.

6.5.3 Interpretation

The investigation pipeline produces, from the unstructured flag set returned by STGAN, a compact subgraph of approximately three hundred nodes that contains the kernel of the attack chain.

The compression ratio between the raw provenance graph and the candidate subgraph is approximately three orders of magnitude, which transforms an unmanageable forensic surface into a target that an analyst can review by hand within minutes. The visualisation conventions chosen for the candidate subgraph make the detection status of each entity legible at a glance and identify the topological role of each entity within the attack chain.

Two limitations of the pipeline are worth stating explicitly. First, the candidate subgraph does not necessarily contain the entry point of the attack: the temporal scoping selects the window of highest anchor density, which may miss the early reconnaissance stage if the attacker operates below the density threshold during reconnaissance. Second, the compactness criterion identifies the smallest of the five candidate windows as the most likely attack neighborhood, but does not currently provide a principled threshold for rejecting all five candidates when the actual attack signal is distributed across multiple windows. A multi-window analyst-review workflow, in which the analyst inspects all five candidates and discards the obvious noise clusters, is a natural extension of the present pipeline and is discussed in the general conclusion as a direction for future work.

These limitations notwithstanding, the present pipeline answers, in the affirmative, the third research question posed in Section 2.3.3: the entities flagged by STGAN can be compressed into a compact subgraph representing the candidate attack neighborhood, using only signals available at inference time. ProvSAGE-C3 therefore provides the practical investigation aid for analyst-driven forensics and, in conjunction with the embedding and architectural contributions reported in the preceding sections, completes the three-contribution structure of this work.

6.6 Synthesis

This final section synthesises the three contributions of this work, identifies the recommended ProvSAGE configuration, and quantifies the gap with the published STGAN baseline.

C1 × C2 interaction matrix. The comparative embedding study (Section 6.3) and the architectural ablation (Section 6.4) are each performed with one axis held constant. The interaction between the two axes is investigated by an additional cross-check in which Word2Vec is paired with the GraphSAGE + Multi-Head Attention architecture. Table 6.6 summarises the resulting interaction matrix.

Table 6.6: Interaction matrix between the embedding choice and the architectural choice on DARPA TRACE, restricted to the two embeddings and the two architectures relevant to ProvSAGE.

	GAT + MHA	GraphSAGE + MHA
Word2Vec	0.9875	**0.9933**
FastText	0.9725	0.9932
Δ (W2V $-$ FastText)	$+1.50$ pt	$+0.01$ pt (noise)

The matrix exhibits an asymmetric interaction. With the GAT backbone, the choice of embedding matters: Word2Vec exceeds FastText by 1.50 F1 points. With the GraphSAGE backbone, the choice of embedding becomes irrelevant within the precision of our measurements: the gap between Word2Vec and FastText shrinks to 0.01 F1 point, which is below the run-to-run variance. The recommended GraphSAGE + Multi-Head Attention architecture is therefore not only the most accurate but also the most robust to upstream design decisions.

Recommended ProvSAGE configuration. The combination Word2Vec + GraphSAGE + Multi-Head Attention reaches an F1 score of **0.9933** on the DARPA TRACE test set, with a parameter count of 387,938 (14 % fewer than the GAT-based STGAN baseline) and a training time of approximately 33 minutes (more than fivefold faster than the GAT-based baseline). This configuration constitutes the recommended ProvSAGE.

Comparison with the published STGAN baseline. The published STGAN baseline reaches an F1 score of 0.9916 on DARPA TRACE under the optimal hyperparameter settings reported in the original paper. The recommended ProvSAGE configuration reaches 0.9933 with fewer parameters and a substantially shorter training time. The detection improvement is modest in absolute terms (0.17 F1 point), but it is obtained at no cost in model size and at a significant gain in training efficiency. The investigation pipeline of Contribution 3 further extends ProvSAGE with a deployable post-detection module that compresses the flag set into a compact attack-neighborhood subgraph.

Closing observation. Taken together, the three contributions of this work identify a STGAN variant that is more accurate, lighter, faster to train, and complemented by an investigation module that is suitable for analyst-driven forensics. ProvSAGE therefore answers the three research questions of Section 2.3 in the affirmative and provides a complete, reproducible package for provenance-graph-based host intrusion detection.

6.7 Conclusion

This chapter has reported the experimental evaluation of the three contributions of this work on the DARPA TRACE dataset. The reproduction results establish the deterministic baseline against which the contributions are compared and document the residual gap with the published figure. The comparative embedding study identifies Word2Vec as the recommended embedding family under the paper-faithful sampling regime. The architectural ablation identifies GraphSAGE + Multi-Head Attention as the recommended architecture, with a 14 % reduction in parameters and a fivefold reduction in training time relative to the GAT-based baseline. The investigation study reports the quantitative output of the ProvSAGE-C3 pipeline and presents a compact candidate attack neighborhood suitable for analyst review. The synthesis identifies Word2Vec + GraphSAGE + Multi-Head Attention as the recommended ProvSAGE configuration, which improves the published STGAN baseline along three independent axes simultaneously.

The next and final chapter consolidates these results into the general conclusion of this thesis.

General Conclusion & Perspectives

This thesis has investigated the design, the evaluation, and the forensic use of STGAN, a recent multi-stream graph neural network for provenance-based host intrusion detection. The work has been motivated by three open questions left by the published evaluation of STGAN: the impact of the semantic embedding family on the detection performance, the impact of the structural and fusion modules on the same performance, and the use of the per-anchor decisions of the detector as the entry point of a forensic investigation. The three contributions of this thesis have addressed these three questions in order.

The first contribution has measured the impact of replacing the Word2Vec embedding of STGAN by FastText, by Sentence-BERT, and by EmbeddingGemma. The four families have been evaluated under a deterministic training protocol on the DARPA TRACE dataset and at two operating points (sub-sampled and full-sampling). The results have shown that the static sub-word embedding of FastText provides a small but reproducible gain over Word2Vec at the sub-sampled operating point, that the contextual transformer embeddings reach the performance of Word2Vec only when conditioned by a task-specific prompt, and that the ordering of the four families inverts between the two operating points. The full-sampling regime restores the Word2Vec embedding to the top of the ranking.

The second contribution has measured the impact of replacing the graph attention layer of STGAN by GraphSAGE and by GCN, and the impact of replacing the multi-head self-attention fusion module by mean pooling and by simple averaging. The complete ablation has identified the combination GraphSAGE plus multi-head self-attention as the configuration that maximises the detection F1 on the DARPA TRACE test set, both on the Kaggle sub-sampled regime and on the station full-sampling regime. The configuration, hereafter named ProvSAGE, reaches an F1 of 0.9933 on the station and outperforms the original STGAN at a fraction of the structural-stream cost.

The third contribution has reframed the per-anchor decisions of STGAN as the entry point of a forensic investigation. A four-stage pipeline has been designed (entity-type filter, multi-window temporal scoping, one-hop neighborhood expansion, compactness ranking) and has been applied to the test set of the engagement. The pipeline has produced a 343-node candidate attack neighborhood that contains six of the malicious nodes of the ground-truth set: four pipes that were

already detected by the per-anchor decisions of STGAN, and two processes that the per-anchor decisions had missed and that the one-hop expansion has recovered through their topological proximity to the four pipes. The candidate neighborhood has been visualized with a CDM18-aware shape and colour scheme that makes the four detected anchors, the two recovered processes, and the surrounding benign context immediately legible to a human analyst.

Several limitations of this work should be acknowledged. The evaluation has been conducted on a single dataset and the generalisation of the findings to other engagements of the Transparent Computing programme remains to be measured. The third contribution has relied on the per-anchor decisions of the original STGAN pipeline, which itself relies on a two-hop forgiveness step that consults the ground-truth set; the development of an equivalent step that does not consult the ground truth is a prerequisite for the deployment of the forensic pipeline outside of the evaluation setting. The four embedding families considered in the first contribution exhaust the families that were available in open-weight form at the time of writing; the evaluation of larger contextual encoders is left for future work.

Several perspectives open at the term of this work. The first perspective concerns the evaluation of ProvSAGE on the other engagements of the DARPA Transparent Computing programme and on the more recent Operationally Transparent Cyber (OpTC) dataset. The second perspective concerns the development of a deployable variant of the two-hop forgiveness step, possibly by replacing the ground-truth consultation by a confidence threshold on the per-anchor scores. The third perspective concerns the integration of the forensic pipeline into an interactive analyst console, in which the candidate attack neighborhood is presented as a navigable graph annotated with the type and the timestamp of every node and edge.

Bibliography

[Sang et al., 2025] Sang, A., Fan, X., Yang, L., Wang, Y., Zhou, L., Jia, J., and Yang, H. (2025). STGAN: Detecting host threats via fusion of spatial-temporal features in host provenance graphs. In *Proceedings of the ACM Web Conference 2025 (WWW)*, pages 1046–1057. https://doi.org/10.1145/3696410.3714925

[Han et al., 2018] Han, X., Pasquier, T., and Seltzer, M. (2018). Provenance-based intrusion detection: Opportunities and challenges. In *Proceedings of the 10th USENIX Workshop on the Theory and Practice of Provenance (TaPP)*, London, UK. https://www.usenix.org/conference/tapp2018/presentation/han

[Bilot et al., 2023] Bilot, T., El Madhoun, N., Al Agha, K., and Zouaoui, A. (2023). Graph neural networks for intrusion detection: A survey. *IEEE Access*, 11:49114–49139. https://doi.org/10.1109/ACCESS.2023.3275789

[Han et al., 2020] Han, X., Pasquier, T., Bates, A., Mickens, J., and Seltzer, M. (2020). UNICORN: Runtime provenance-based detector for advanced persistent threats. In *Proceedings of the 2020 Network and Distributed System Security Symposium (NDSS)*. https://doi.org/10.14722/ndss.2020.24046

[Milajerdi et al., 2019] Milajerdi, S. M., Gjomemo, R., Eshete, B., Sekar, R., and Venkatakrishnan, V. N. (2019). HOLMES: Real-time APT detection through correlation of suspicious information flows. In *Proceedings of the 2019 IEEE Symposium on Security and Privacy (S&P)*, pages 1137–1152. https://doi.org/10.1109/SP.2019.00026

[Gehani and Tariq, 2012] Gehani, A. and Tariq, D. (2012). SPADE: Support for provenance auditing in distributed environments. In *Proceedings of the 13th International Middleware Conference*, pages 101–120. https://doi.org/10.1007/978-3-642-35170-9_6

[Pasquier et al., 2017] Pasquier, T., Han, X., Goldstein, M., Moyer, T., Eyers, D., Seltzer, M., and Bacon, J. (2017). Practical whole-system provenance capture. In *Proceedings of the 2017 Symposium on Cloud Computing (SoCC)*, pages 405–418. https://doi.org/10.1145/3127479.3129249

[Hossain et al., 2020] Hossain, M. N., Sheikha Mehrabadi, S., Abu-Ghazaleh, N., Mukherjee,

B., and Sekar, R. (2020). Combating dependence explosion in forensic analysis using alternative tag propagation semantics. In *Proceedings of the 2020 IEEE Symposium on Security and Privacy (S&P)*, pages 1139–1155. https://doi.org/10.1109/SP40000.2020.00064

[Kwon et al., 2018] Kwon, Y., Wang, F., Wang, W., Lee, K. H., Lee, W.-C., Ma, S., Zhang, X., Xu, D., Jha, S., Ciocarlie, G., Gehani, A., and Yegneswaran, V. (2018). MCI: Modeling-based causality inference in audit logging for attack investigation. In *Proceedings of the 2018 Network and Distributed System Security Symposium (NDSS)*. https://doi.org/10.14722/ndss.2018.23306

[Yang et al., 2023] Yang, F., Xu, J., Xiong, C., Li, Z., and Zhang, K. (2023). ProGrapher: An anomaly detection system based on provenance graph embedding. In *Proceedings of the 32nd USENIX Security Symposium*, pages 3405–3422, Anaheim, CA. https://www.usenix.org/conference/usenixsecurity23/presentation/yang-fan

[Hamilton et al., 2017] Hamilton, W. L., Ying, R., and Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30. https://proceedings.neurips.cc/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7ebea9-Abstract.html

[Veličković et al., 2018] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph attention networks. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=rJXMpikCZ

[Grover and Leskovec, 2016] Grover, A. and Leskovec, J. (2016). node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 855–864. https://doi.org/10.1145/2939672.2939754

[Narayanan et al., 2017] Narayanan, A., Chandramohan, M., Venkatesan, R., Chen, L., Liu, Y., and Jaiswal, S. (2017). graph2vec: Learning distributed representations of graphs. In *Proceedings of the International Workshop on Mining and Learning with Graphs (MLG)*. https://arxiv.org/abs/1707.05005

[Wang et al., 2022] Wang, S., Wang, Z., Zhou, T., Yin, X., Han, D., Zhang, H., Sun, H., Shi, X., and Yang, J. (2022). ThreaTrace: Detecting and tracing host-based threats in node level through provenance graph learning. *IEEE Transactions on Information Forensics and Security*, 17:3972–3987. https://doi.org/10.1109/TIFS.2022.3208815

[Ur Rehman et al., 2024] Ur Rehman, M., Ahmadi, H., and Ul Hassan, W. (2024). FLASH: A comprehensive approach to intrusion detection via provenance graph representation learning. In *Proceedings of the 2024 IEEE Symposium on Security and Privacy (S&P)*, pages 139–139. https://doi.org/10.1109/SP54263.2024.00139

[Wang et al., 2025] Wang, L., Fang, L., and Hu, Y. (2025). A dynamic provenance graph-based detector for advanced persistent threats. *Expert Systems with Applications*, 265:125891. https://doi.org/10.1016/j.eswa.2024.125891

[Paudel and Huang, 2022] Paudel, R. and Huang, H. H. (2022). Pikachu: Temporal walk based dynamic graph embedding for network anomaly detection. In *Proceedings of the IEEE/IFIP Network Operations and Management Symposium (NOMS)*. https://doi.org/10.1109/NOMS54207.2022.9789905

[King and Huang, 2023] King, I. J. and Huang, H. Howie (2023). Euler: Detecting network lateral movement via scalable temporal link prediction. *ACM Transactions on Privacy and Security*, 26(3):1–31. https://doi.org/10.1145/3588771

[Reha et al., 2023] Reha, J., Russo, M., Lovisotto, G., Gravina, A., and Grohnfeldt, C. (2023). Anomaly detection in continuous-time temporal provenance graphs. In *Temporal Graph Learning Workshop at NeurIPS 2023*. https://openreview.net/forum?id=88tGIxxhsf

[Rossi et al., 2020] Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., and Bronstein, M. (2020). Temporal graph networks for deep learning on dynamic graphs. In *ICML 2020 Workshop on Graph Representation Learning*. https://arxiv.org/abs/2006.10637

[Mikolov et al., 2013] Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., and Dean, J. (2013). Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26. https://proceedings.neurips.cc/paper/2013/hash/9aa42b31882ec039965f3c4923ce901b-Abstract.html

[Bojanowski et al., 2017] Bojanowski, P., Grave, E., Joulin, A., and Mikolov, T. (2017). Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5:135–146. https://doi.org/10.1162/tacl_a_00051

[Devlin et al., 2019] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pages 4171–4186. https://doi.org/10.18653/v1/N19-1423

[Reimers and Gurevych, 2019] Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992. https://doi.org/10.18653/v1/D19-1410

[Gemma Team, Google DeepMind, 2024] Gemma Team, Google DeepMind (2024). Gemma: Open models based on gemini research and technology. Technical report, Google DeepMind. https://arxiv.org/abs/2403.08295

[Kipf and Welling, 2017] Kipf, T. N. and Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=SJU4ayYgl

[Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[Li et al., 2024] Li, H., Jian, Z., Pan, S., Zhang, J., Chen, F., and Yang, M. (2024). NODLINK: An online system for fine-grained APT attack detection and investigation. In *Proceedings of the 2024 IEEE Symposium on Security and Privacy (S&P)*. https://doi.org/10.1109/SP54263.2024.00085